



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 3 Combinational Logic



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Magnitude Comparators

2 Signed Binary Numbers & Arithmetic

3 Decoders

4 Codes & Conversions

1. Magnitude Comparators

Main Content

1. Learning Outcomes

2. Multi-Bit Comparator Design

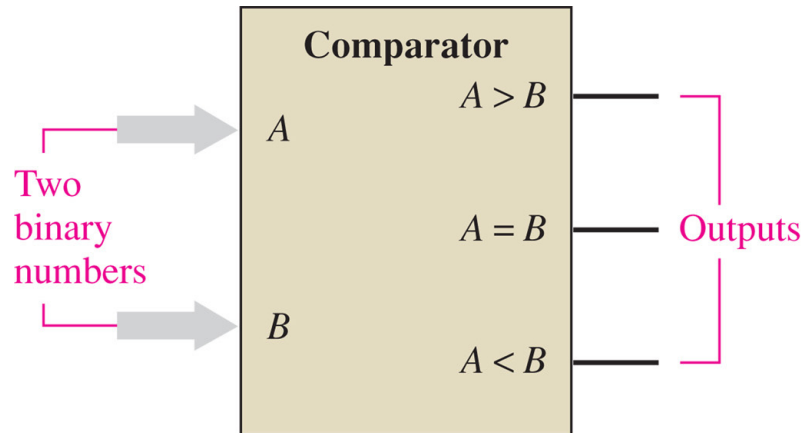
1. Learning Outcomes

1. Design a multi-bit magnitude comparator using a bit-wise approach (not the truth table approach)
2. Implement a subtractor/comparator using an adder
3. Determine the 1's and 2's complements of a binary number
4. Express signed binary numbers in 1's complement & 2's complement
5. Implement addition/subtraction of signed binary numbers
6. Describe various decoder blocks
7. Use a decoder to implement a logic function (Full Adder)
8. Describe codes including binary coded decimal (BCD) & Gray codes

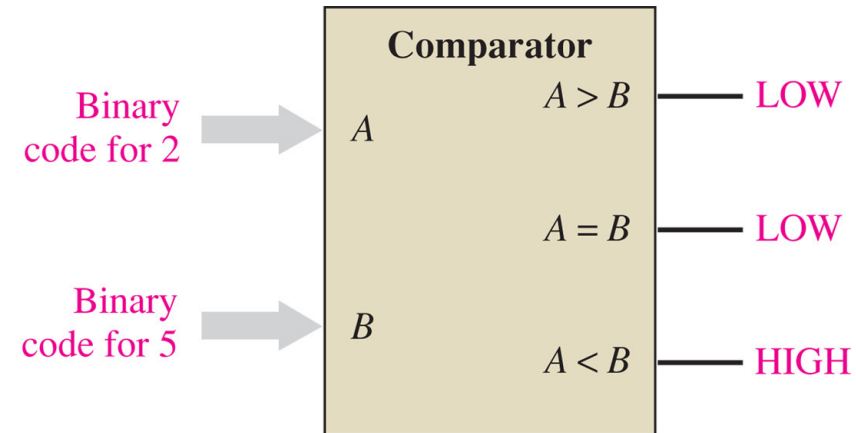
1. Magnitude Comparators

2. Multi-Bit Comparator Design

(1) Introduction to magnitude comparators ($A > B$, $A = B$, $A < B$)



(a) Basic magnitude comparator

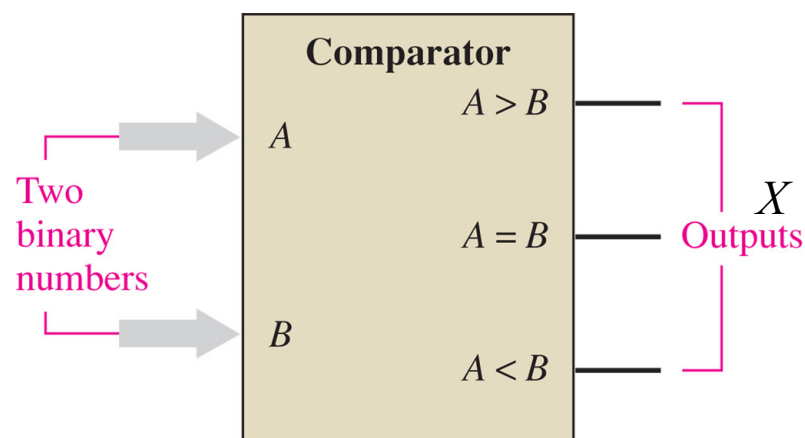


(b) Example: A is less than B ($2 < 5$) as indicated by the HIGH output ($A < B$)

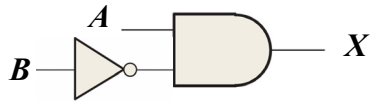
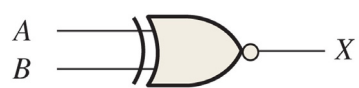
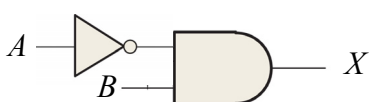
1. Magnitude Comparators

(2) 1-bit comparator truth table and Boolean expressions

A and *B* each 1-bit number
Truth table for each
output:



(a) Basic magnitude comparator

<i>A</i>	<i>B</i>	<i>A > B</i>	Boolean Expression
0	0	0	$X = A \bar{B}$
0	1	0	
1	0	1	
1	1	0	
<i>A</i>	<i>B</i>	<i>A = B</i>	Boolean Expression
0	0	1	$X = \bar{A} \bar{B} + A B$
0	1	0	
1	0	0	
1	1	1	
<i>A</i>	<i>B</i>	<i>A < B</i>	Boolean Expression
0	0	0	$X = \bar{A} B$
0	1	1	
1	0	0	
1	1	0	

(3) 4-bit Magnitude Comparator

EXAMPLE: Design a circuit that compares the magnitudes of 2 binary numbers, each having 4 bits.

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

$$A >, =, < B ?$$

STEP 1: Capture circuit behavior (input/output relation) in a truth table (skip this step if you can directly write the Boolean expression)

STEP 2: Write the Boolean expression from the truth table (or directly)

STEP 3: Simplify the Boolean expression (skip for now)

STEP 4: Build the circuit using basic logic gates (as stated in the Boolean expression (i.e., draw the gate schematic of the circuit))

This would require 8-input truth table: $2^8 = 256$ rows!

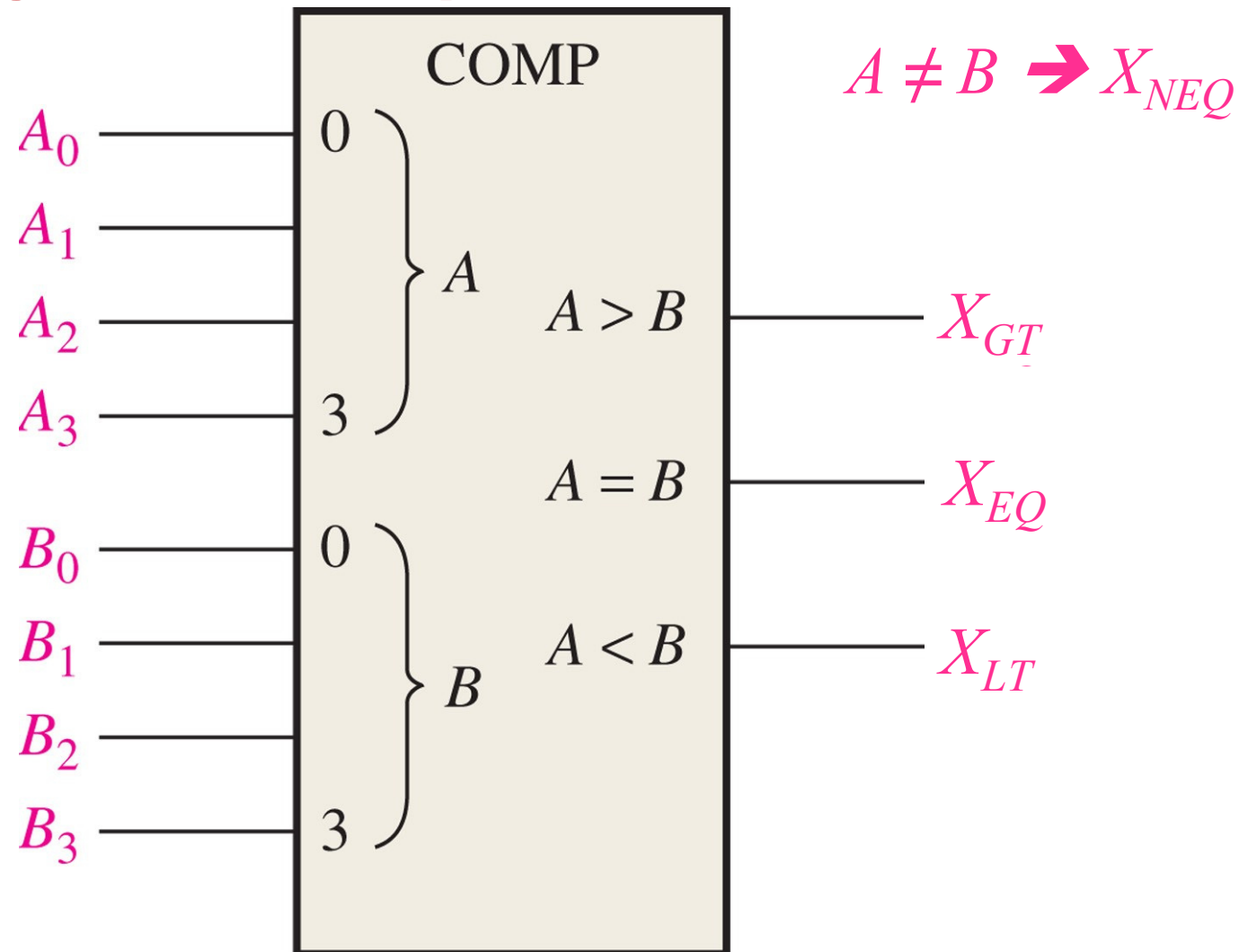
NOTE: Truth table is NOT practical for too many inputs (more than four)

For those cases, perform a bit-wise operation (comparison) using a known combinational logic block (1-bit comparator)

1. Magnitude Comparators



(3) 4-bit Magnitude Comparator



1. Magnitude Comparators

(3) 4-bit Magnitude Comparator

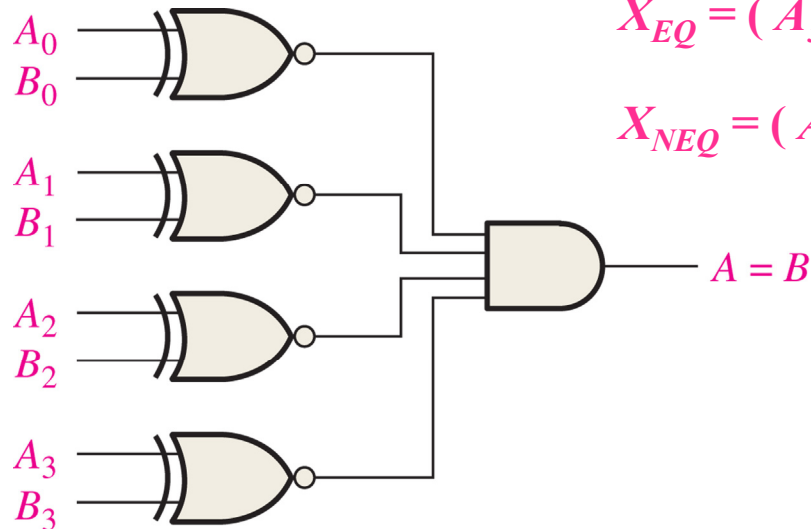
Start from MSB pairs comparing the bit pair and move to lower order bits

$A = B$ ($X_{EQ} = 1$) if “all” the corresponding bit pairs are equal

$$X_{EQ} = (A_3 = B_3) \text{ AND } (A_2 = B_2) \text{ AND } (A_1 = B_1) \text{ AND } (A_0 = B_0)$$

$A \neq B$ ($X_{NEQ} = 1$) if “any” of the corresponding bit pairs differs

$$X_{NEQ} = (A_3 \neq B_3) \text{ OR } (A_2 \neq B_2) \text{ OR } (A_1 \neq B_1) \text{ OR } (A_0 \neq B_0)$$



$$X_{EQ} = \overline{(A_3 \oplus B_3)} \overline{(A_2 \oplus B_2)} \overline{(A_1 \oplus B_1)} \overline{(A_0 \oplus B_0)}$$

$$X_{NEQ} = (A_3 \oplus B_3) + (A_2 \oplus B_2) + (A_1 \oplus B_1) + (A_0 \oplus B_0)$$

$$X_{NEQ} = \overline{X_{EQ}}$$

(3) 4-bit Magnitude Comparator

Start from MSB pairs comparing the bit pair and move to lower order bits

$A > B$ ($X_{GT} = 1$)

$$X_{GT} = (A_3 > B_3) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 > B_2) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 = B_2) \text{ AND } (A_1 > B_1) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 = B_2) \text{ AND } (A_1 = B_1) \text{ AND } (A_0 > B_0)$$

$$X_{GT} = (A_3 \bar{B}_3) +$$

$$(\overline{A_3 \oplus B_3}) (A_2 \bar{B}_2) +$$

$$(\overline{A_3 \oplus B_3}) (\overline{A_2 \oplus B_2}) (A_1 \bar{B}_1) +$$

$$(\overline{A_3 \oplus B_3}) (\overline{A_2 \oplus B_2}) (\overline{A_1 \oplus B_1}) (A_0 \bar{B}_0)$$

(3) 4-bit Magnitude Comparator

Start from MSB pairs comparing the bit pair and move to lower order bits

$$A < B (X_{LT} = 1)$$

$$X_{LT} = (A_3 < B_3) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 < B_2) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 = B_2) \text{ AND } (A_1 < B_1) \text{ OR}$$

$$(A_3 = B_3) \text{ AND } (A_2 = B_2) \text{ AND } (A_1 = B_1) \text{ AND } (A_0 < B_0)$$

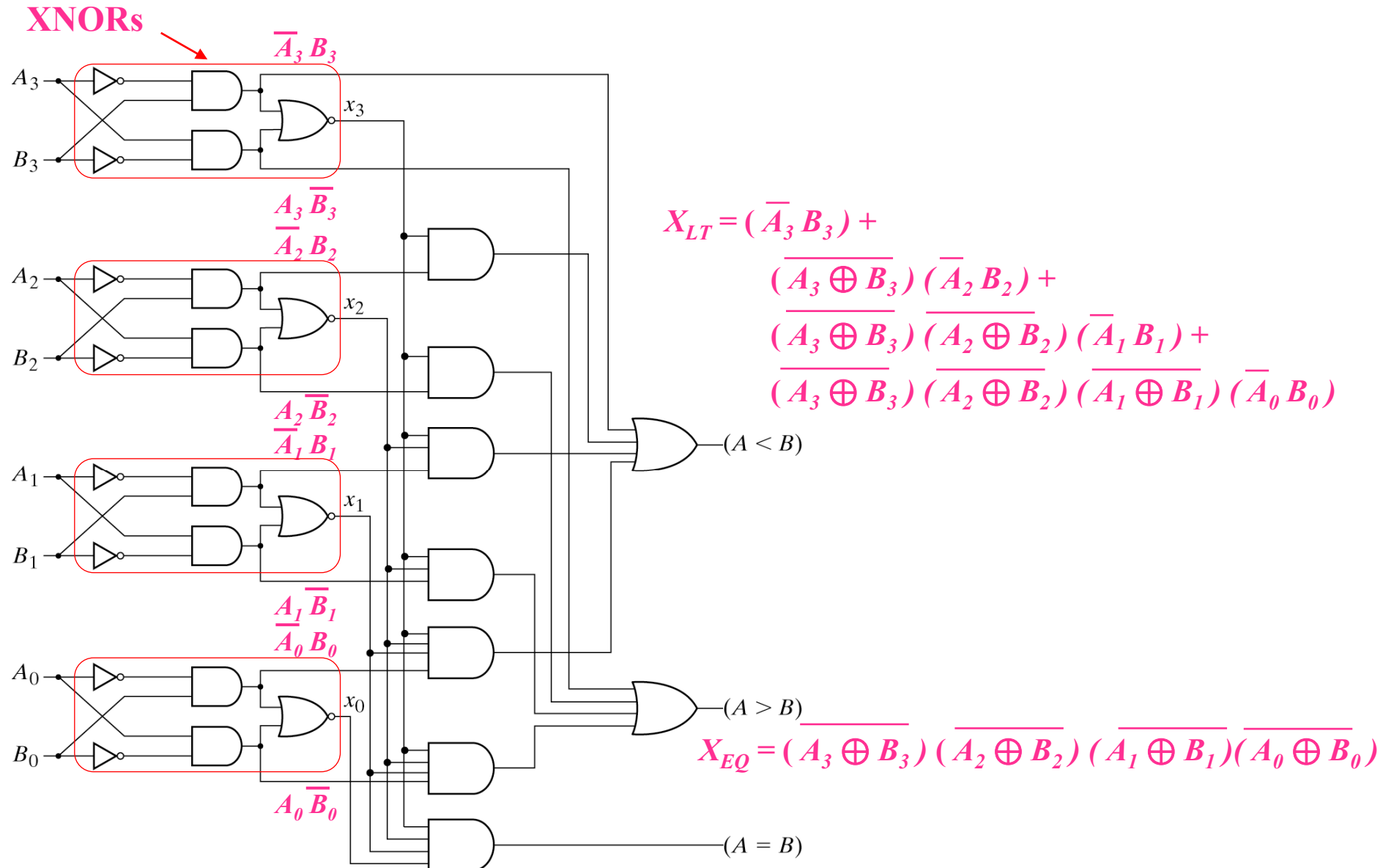
$$X_{LT} = (\bar{A}_3 B_3) +$$

$$(\overline{A_3 \oplus B_3}) (\bar{A}_2 B_2) +$$

$$(\overline{A_3 \oplus B_3}) (\overline{A_2 \oplus B_2}) (\bar{A}_1 B_1) +$$

$$(\overline{A_3 \oplus B_3}) (\overline{A_2 \oplus B_2}) (\overline{A_1 \oplus B_1}) (\bar{A}_0 B_0)$$

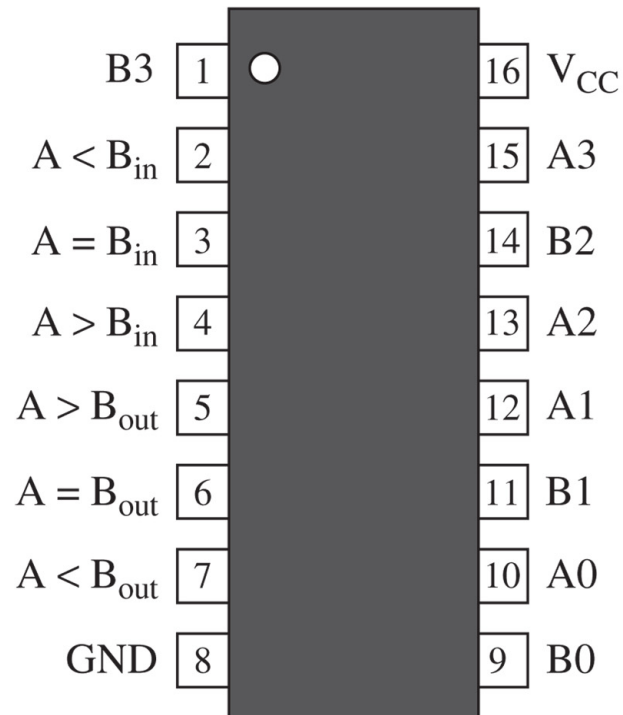
1. Magnitude Comparators



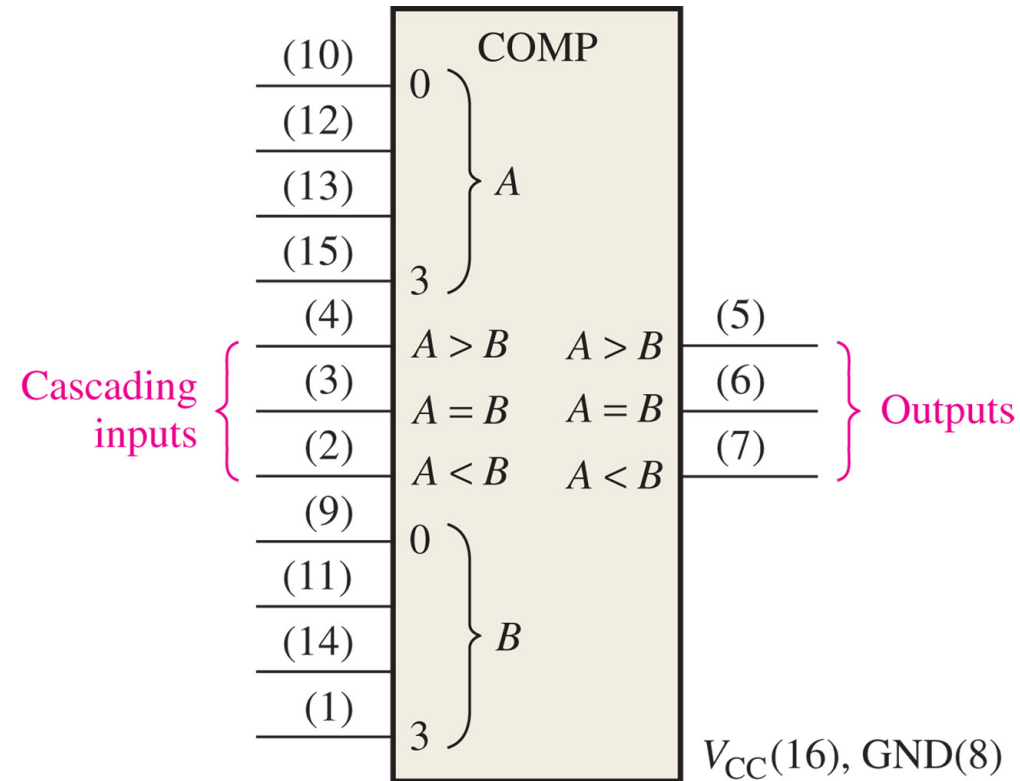
1. Magnitude Comparators

(4) IC implementation (74HC85) and cascading for 8-bit comparisons

The 74HC85/74LS85 4-bit magnitude comparator



(a) Pin diagram

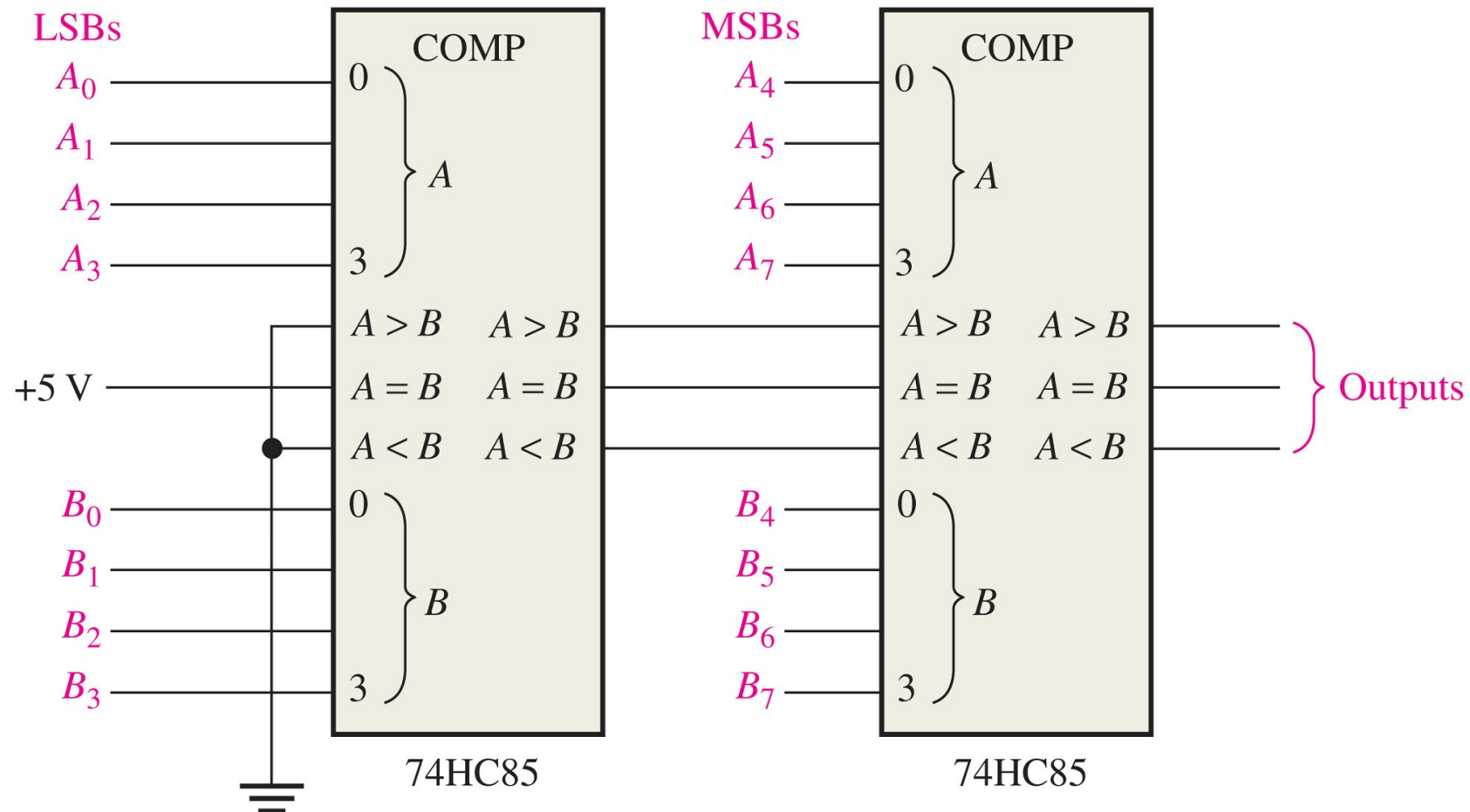


(b) Logic symbol

1. Magnitude Comparators

(4) IC implementation (74HC85) and cascading for 8-bit comparisons

using two 74HC85s



2. Signed Binary Numbers & Arithmetic

Main Content

1. Complement Systems
2. Arithmetic Operations

2. Signed Binary Numbers & Arithmetic

4-bit Adder/Subtractor/Comparator

Compare A and B by
doing subtraction

$$S = A - B$$

If $S > 0$ then $A > B$

If $S = 0$ then $A = B$

If $S < 0$ then $A < B$

Subtraction can be
done by addition
(using an adder) if
negative B is applied:

$$S = A + (-B)$$

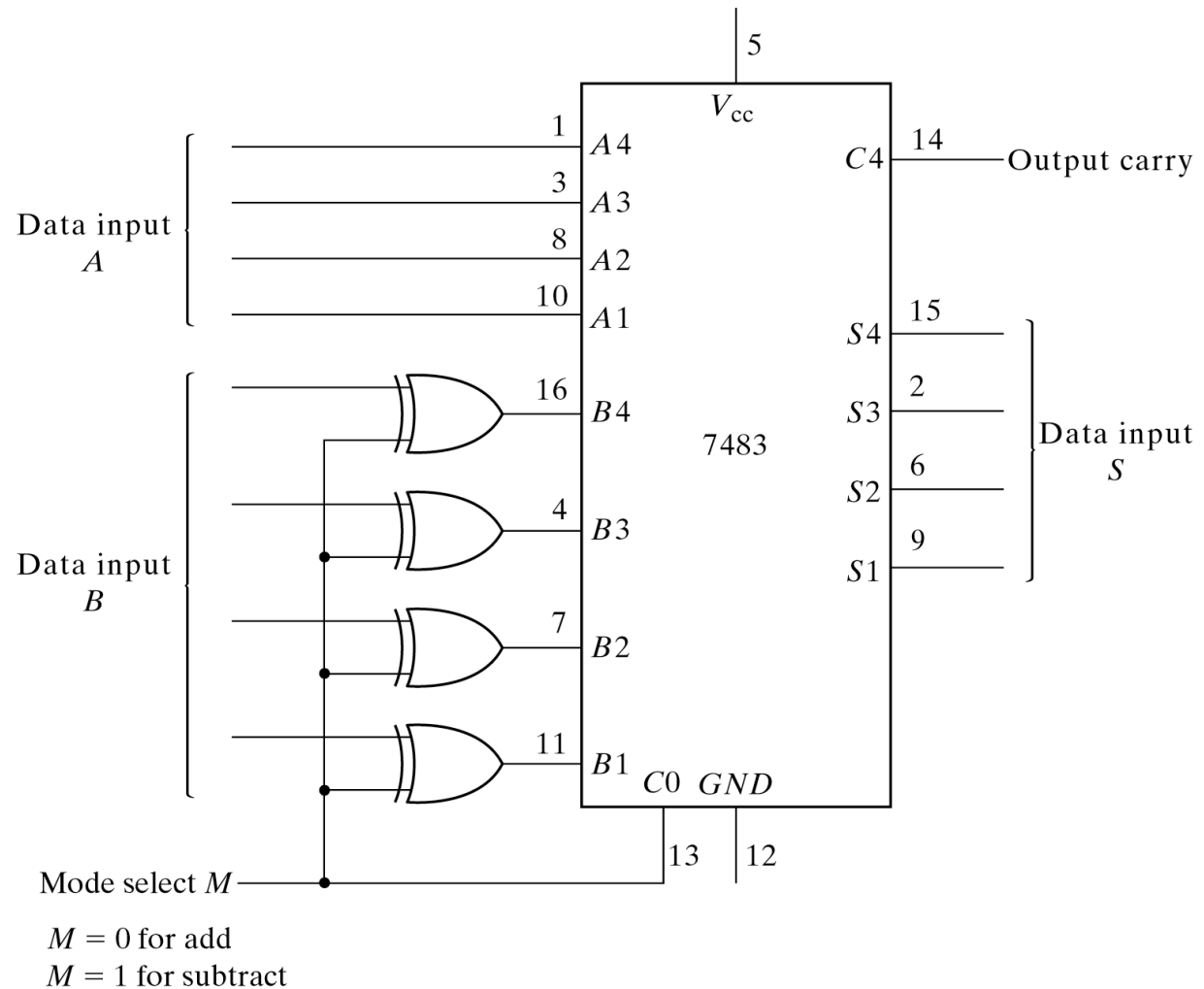


Fig. 11-11 4-Bit Adder-Subtractor

1's complement definition and example

A and B n -bit binary numbers and A is all 1s

$$S = A + (-B)$$

then

$$S = \overline{B}$$

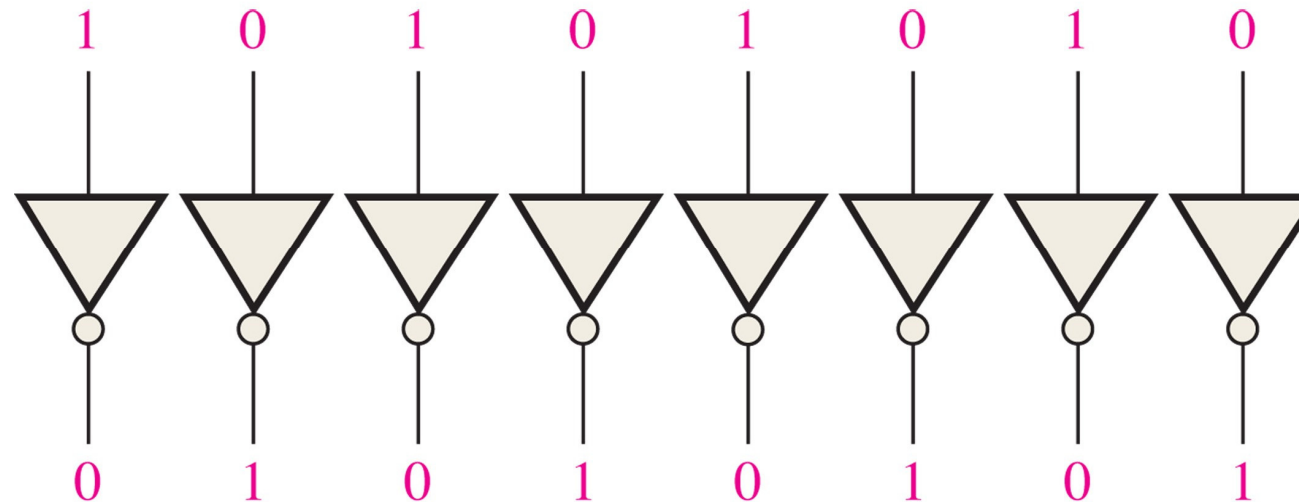
S is called 1's complement of B because S complements B to all 1s.

$A: 11111111$

— $B: 10101010$

$S: 01010101$

So just invert all the bits!



*Taking 1's complement of a binary number
(inverting all the bits) performs subtraction*

2's complement calculation (invert + add 1)



A and B n -bit numbers and A is made of all 1s

$$S = A + 1 + (-B)$$

then

$$S = \bar{B} + 1$$

S is called 2's complement of B because it complements B to $A + 1$, which is the binary equivalent of 2^n .

For $n = 8$, the binary equivalent of 2^8 is 100000000.

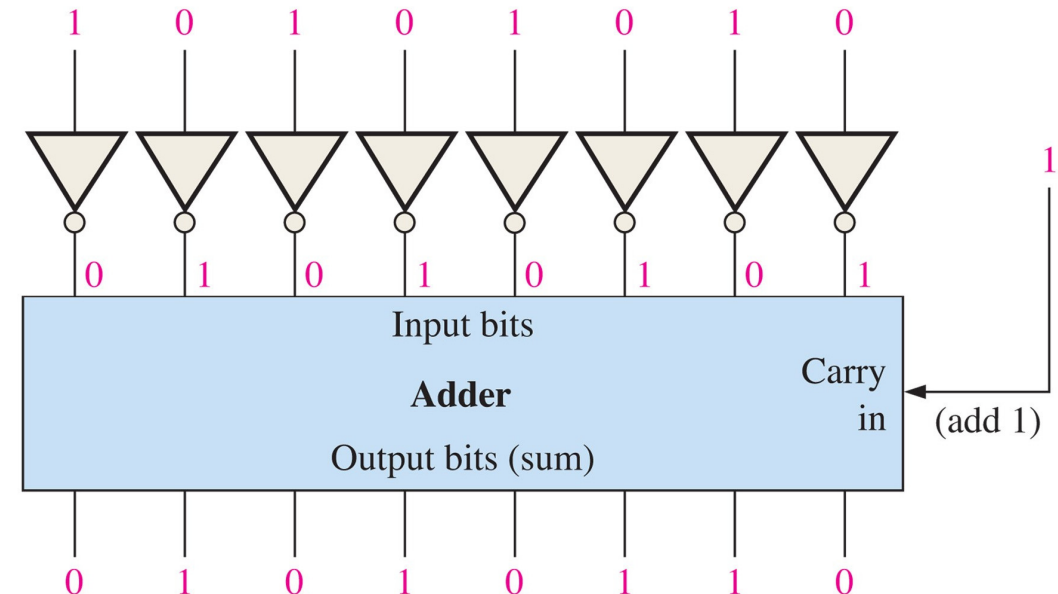
Shortcut: Invert all the bits and then add 1.

A : 11111111

$A + 1$: 100000000

— B : 10101010

S : 01010110



Signed binary numbers

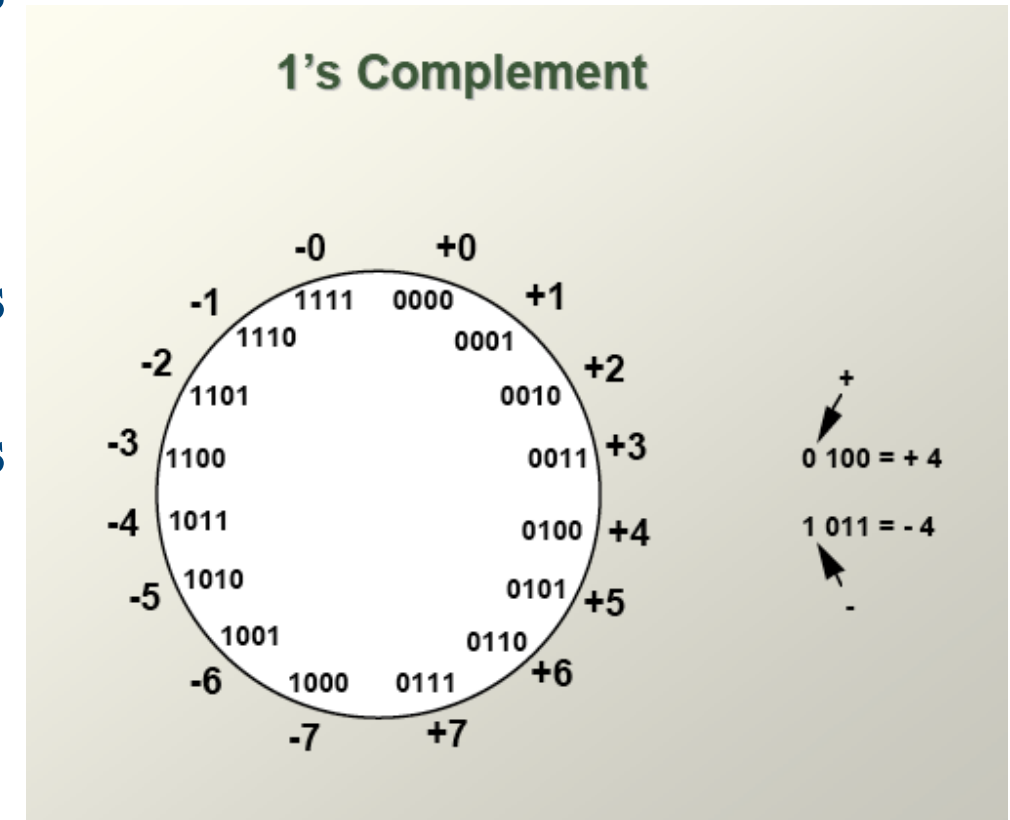
In both 1's and 2's complements, MSB represents the sign:

- 0 for positive binary numbers
- 1 for negative binary numbers

For negative binary numbers, the magnitude is given by 1's or 2's complement of the number.

For positive binary numbers, the magnitude is same as the unsigned case.

Example: In 1's complement system
1011 is a negative binary number
and represents (–0100) which is –4.
0100 is a positive number (+ 4)



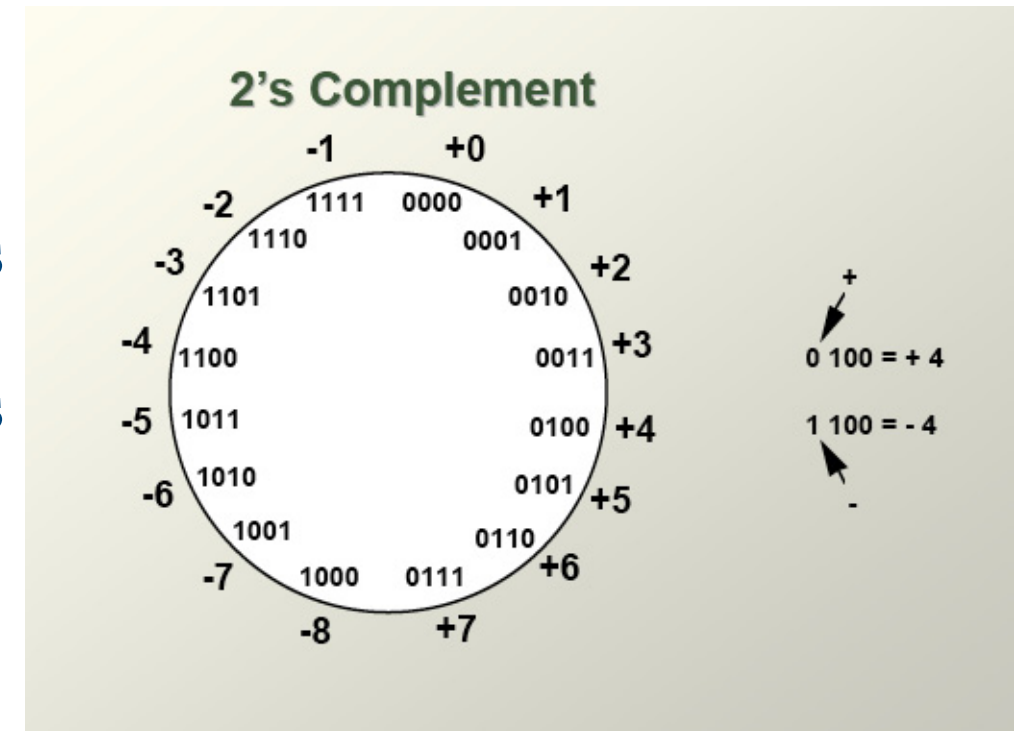
In both 1's and 2's complements, MSB represents the sign:

- 0 for positive binary numbers
- 1 for negative binary numbers

For negative binary numbers, the magnitude is given by 1's or 2's complement of the number.

For positive binary numbers, the magnitude is same as the unsigned case.

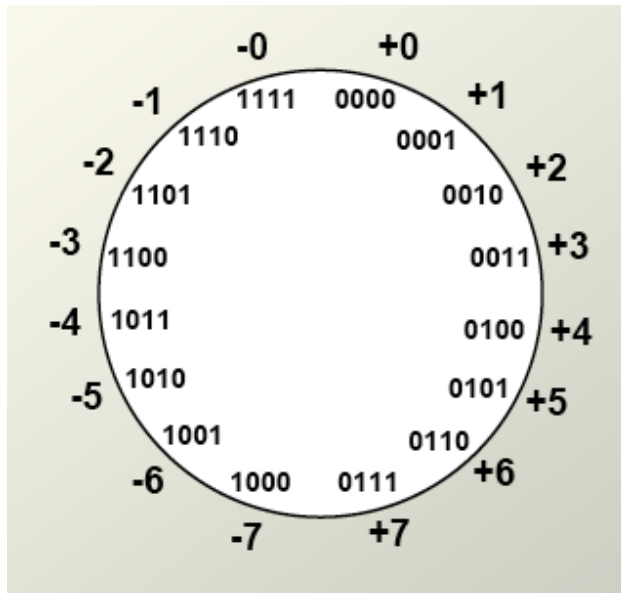
Example: In 2's complement system
1100 is a negative binary number
and represents (−0100) which is −4.
0100 is a positive number (+ 4).



Problem with 1's complement

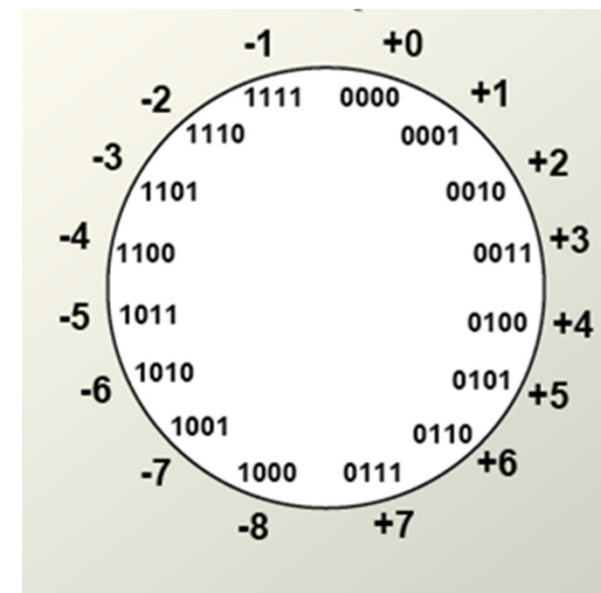
1's complement

Two zeros : positive and negative zeros



2's complement

Only one zero



That's why 2's complement is popular

Sign-magnitude vs. complement systems



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

In sign-and-magnitude numbers, MSB also represents the sign:

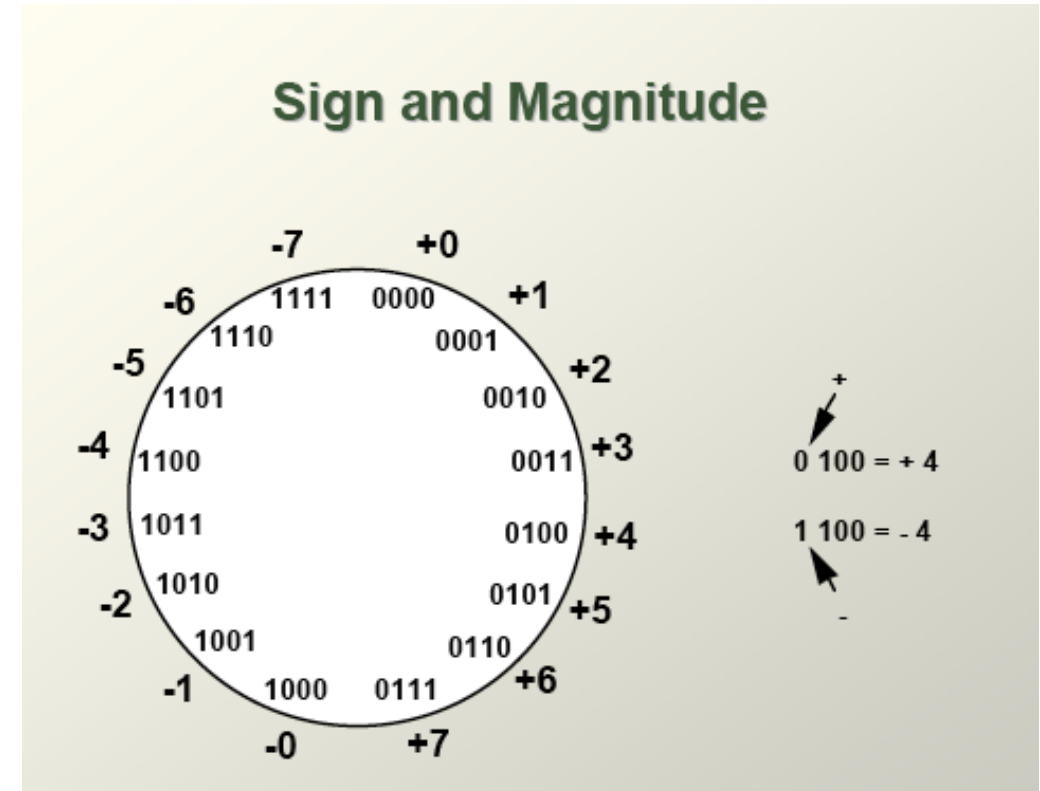
- 0 for positive binary numbers
- 1 for negative binary numbers

but MSB does not determine the magnitude.

Example:

1100 is a negative binary number and represents (-100) which is -4 .
0100 is a positive number $(+4)$.

Good to know but we won't use it.



2. Signed Binary Numbers & Arithmetic

Example of ± 19 in different representations

Express +19 and -19 as 8-bit numbers in sign-magnitude, 1's complement, and 2's complement

TABLE 2-10

	Sign-Magnitude	1's Comp	2's Comp
+19	<i>00010011</i>	<i>00010011</i>	<i>00010011</i>
-19	<i>10010011</i>	<i>11101100</i>	<i>11101101</i>

2. Arithmetic Operations

Signed binary numbers

- In the 1's complement form, a negative number is the 1's complement of the corresponding positive number.
- In the 2's complement form, a negative number is the 2's complement of the corresponding positive number.

2. Signed Binary Numbers & Arithmetic

(1) Addition with overflow handling

We will use only 2's complement form which is more common

It can be also done using 1's complement form with slight variation

The addend and the augend refer to the two numbers to be added.

The sum is the result.

Four cases encountered in adding two signed binary numbers

1. Both numbers positive
2. Positive number with magnitude larger than negative number
3. Negative number with magnitude larger than positive number
4. Both numbers negative

Following examples use 8-bit signed numbers.

Add the two numbers and discard any final carry bit.

2. Signed Binary Numbers & Arithmetic

(1) Addition with overflow handling

Addition of two positive numbers yields a positive number

Both numbers positive:

$$\begin{array}{r} 00000111 \quad 7 \\ + 00000100 \quad + 4 \\ \hline 00001011 \quad 11 \end{array}$$

The sum is positive and is therefore in true (uncomplemented) binary.

Addition of a positive number and a smaller negative number yields a positive number

Positive number with magnitude larger than negative number:

$$\begin{array}{r} 00001111 \quad 15 \\ + 11111010 \quad + -6 \\ \hline 1 \quad 00001001 \quad 9 \end{array}$$

Discard carry $\longrightarrow$ 1

The final carry bit is discarded. The sum is positive and therefore in true (uncomplemented) binary.

2. Signed Binary Numbers & Arithmetic

(1) Addition with overflow handling

Addition of a positive number and a larger negative number yields a negative number in 2's complement

Negative number with magnitude larger than positive number:

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \quad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

The sum is negative and therefore in 2's complement form.

Addition of two negative numbers yields a negative number in 2's complement

Both numbers negative:

$$\begin{array}{r} 11111011 \\ + 11110111 \\ \hline 11110010 \end{array} \quad \begin{array}{r} -5 \\ + -9 \\ \hline -14 \end{array}$$

Discard carry $\longrightarrow$ 1

The final carry bit is discarded. The sum is negative and therefore in 2's complement form.

2. Signed Binary Numbers & Arithmetic

(2) Subtraction using 2' s complement

Subtraction is a special case of addition.

Example: subtract +6 (the subtrahend) from + 9 (the minuend)

This is equivalent to adding $- 6$ to $+ 9$.

$$9 - 6 = 9 + (- 6)$$

Subtraction is addition with the sign of the subtrahend changed.

Change the sign of the subtrahend and adds it to the minuend.

Use 2's complement to change the sign of a positive or negative number

To subtract two signed numbers, take the 2's complement of the subtrahend and then add. Discard any final carry bit.

Important: both numbers must have the same number of bits.

2. Signed Binary Numbers & Arithmetic



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

(2) Subtraction using 2's complement

(a) In this case, $8 - 3 = 8 + (-3) = 5$.

	00001000	Minuend (+8)
	+ 11111101	2's complement of subtrahend
Discard carry →	<u>1 0000101</u>	Difference (+5)

(b) In this case, $12 - (-9) = 12 + 9 = 21$.

00001100	Minuend (+12)
+ 00001001	2's complement of subtrahend
<u>00010101</u>	Difference (+21)

2. Signed Binary Numbers & Arithmetic

(2) Subtraction using 2's complement

(c) In this case, $-25 - (+19) = -25 + (-19) = -44$.

	11100111	Minuend (-25)
	+ 11101101	2's complement of subtrahend
Discard carry	<u>1 11010100</u>	Difference (-44)

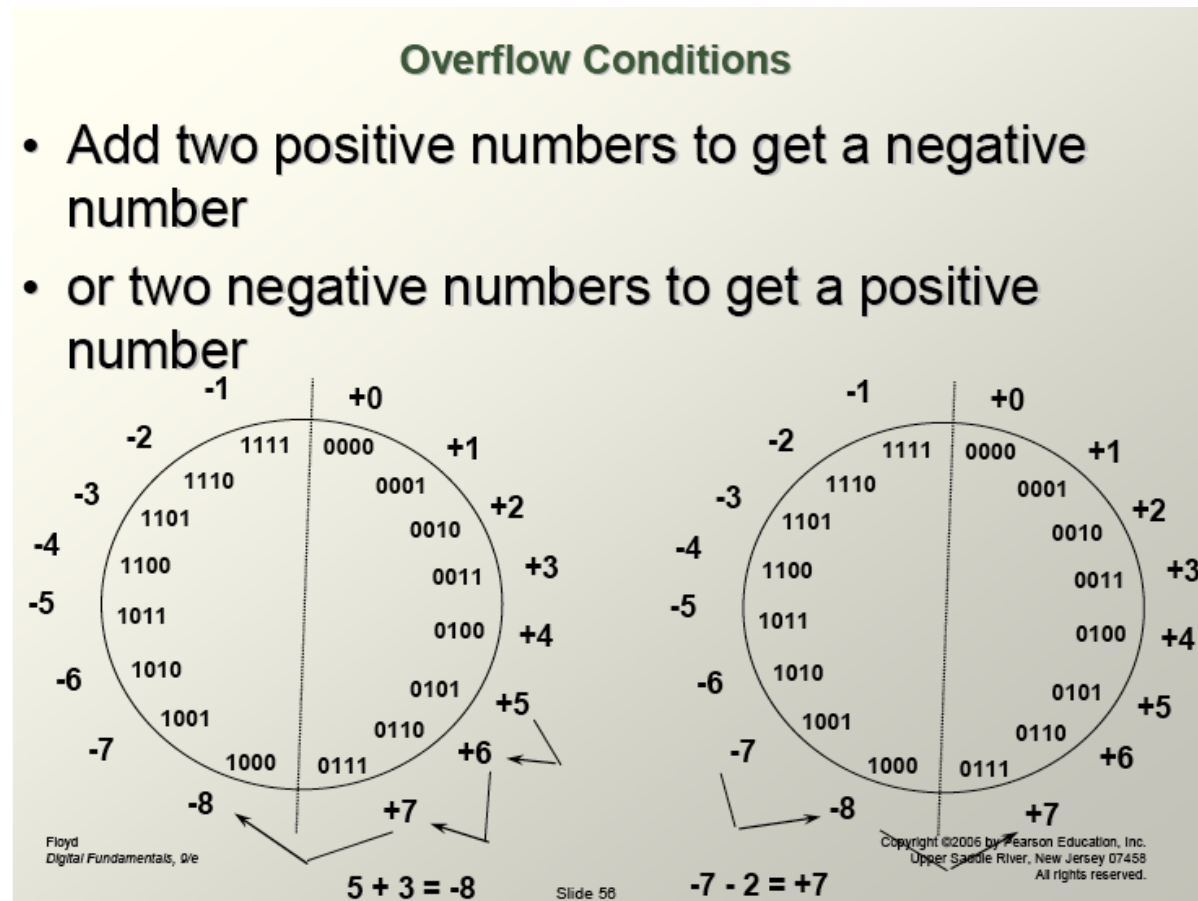
(d) In this case, $-120 - (-30) = -120 + 30 = -90$.

	10001000	Minuend (-120)
	+ 00011110	2's complement of subtrahend
	<u>10100110</u>	Difference (-90)

2. Signed Binary Numbers & Arithmetic

(3) Overflow detection and example

Below examples: you get wrong answers if you use only 4-bit signed numbers because correct answers are not within the 4-bit number set. Correct answers lie in 5-bit number set so you have to use 5-bit numbers



4-bit Adder/Subtractor/Comparator

A 2-input XOR acts as an inverter when one of the inputs is set to 1

It acts as a buffer otherwise

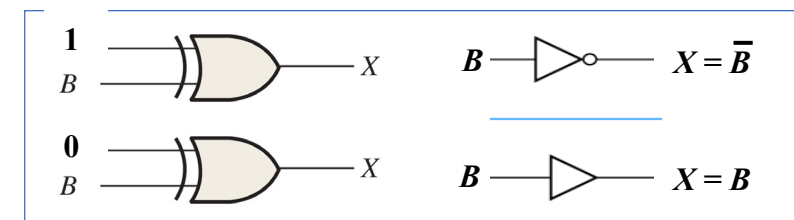
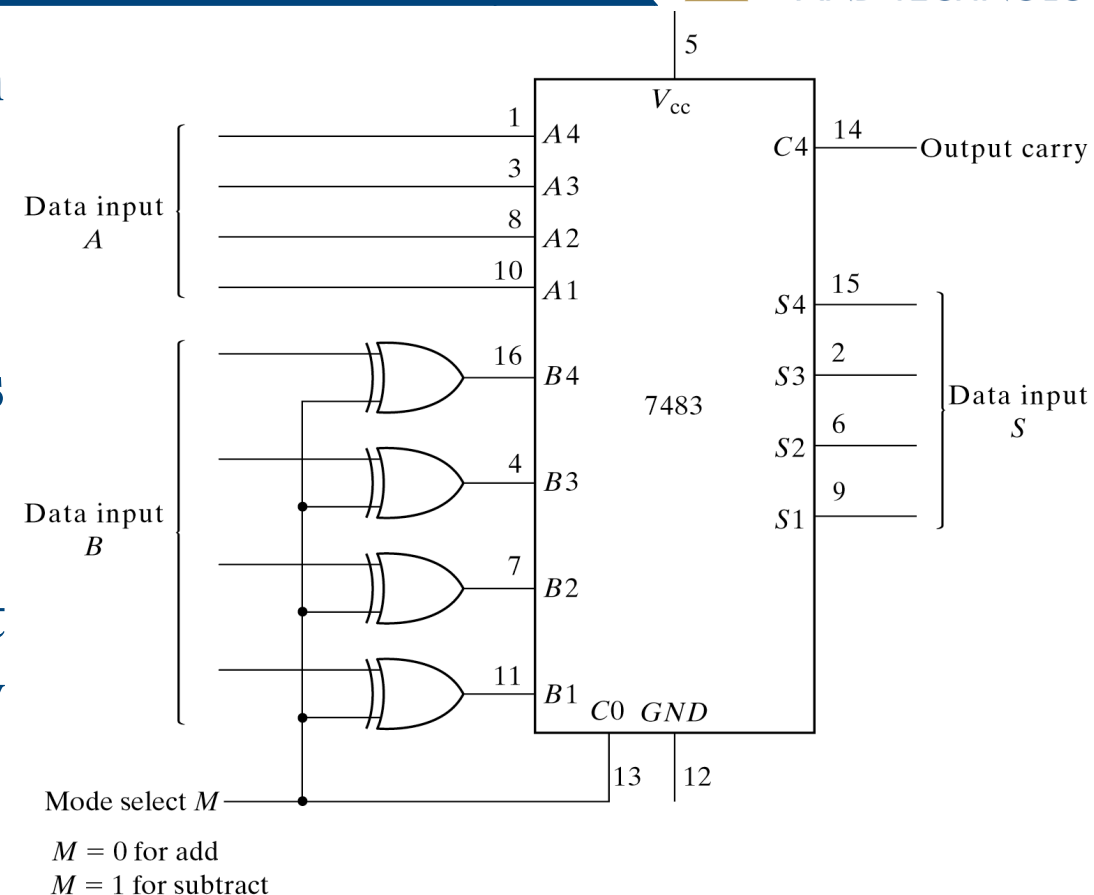
Thus, XOR array when $M = 1$ produces 1's complement of B

7483 then convert it into 2's complement by adding 1 internally ($M = 1$ is also carry input)

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

$X = B$; buffer

$X = \bar{B}$; inverter

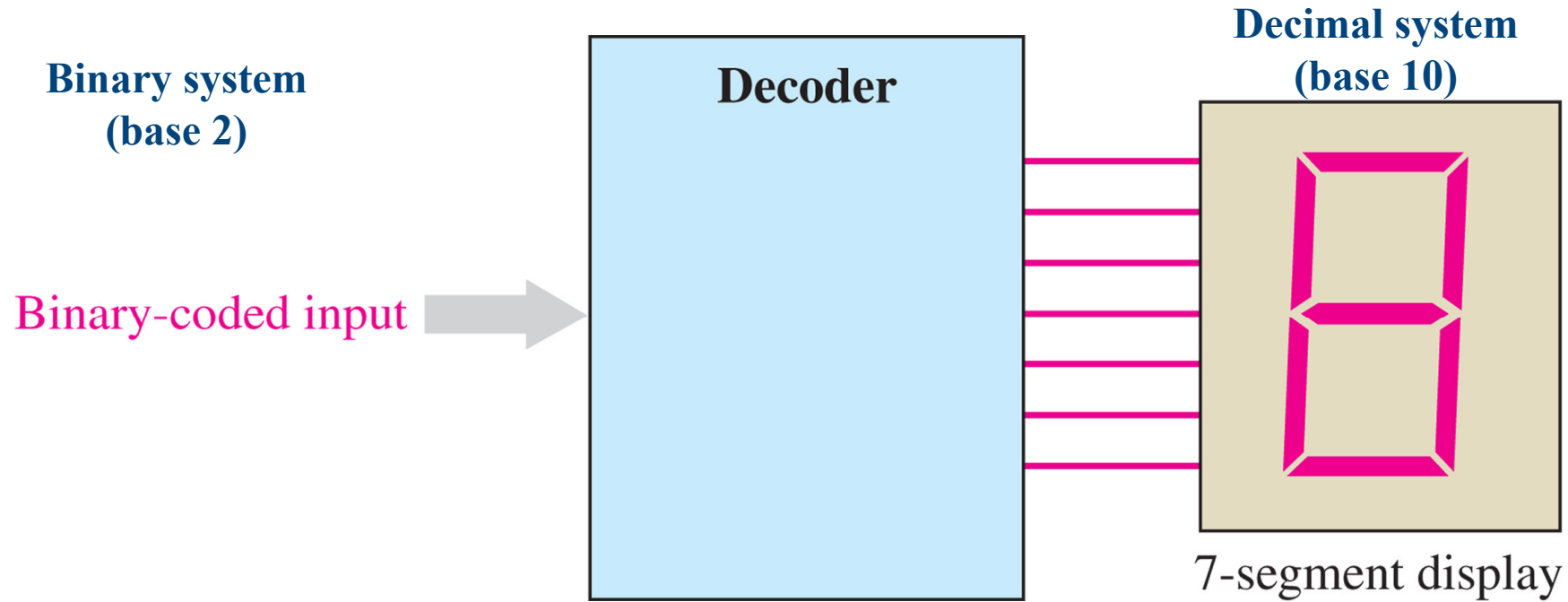


Main Content

1. Basic Decoders

2. Specialized Decoders

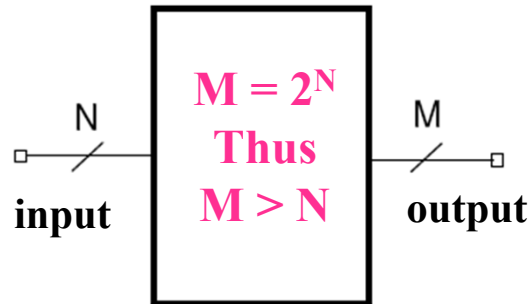
1. Basic Decoders



Interfacing digital circuits to outside world

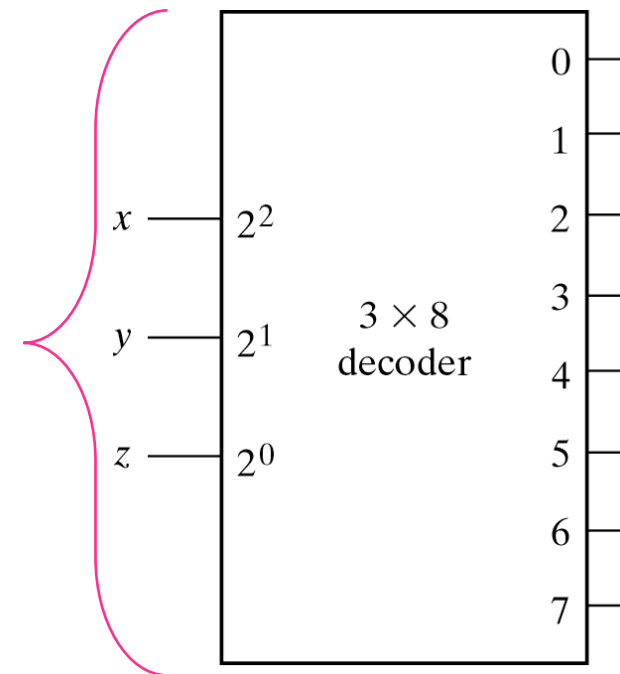
(1) 3-to-8 line decoder

*Generic Decoder with
N-bit input and M-bit output*



Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

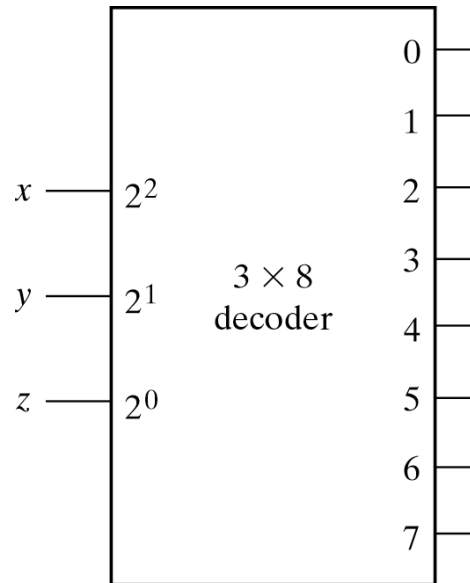
- The output terminals are “active HIGH” (no bubbles)
Meaning the output asserted 1 “HIGH” is activated
- All the outputs are asserted 0 “LOW” except the one corresponds to decimal equivalent of the applied binary input (see the Truth Table)
So decoding from binary to decimal
- It generates “one-hot” code at the output terminals
So also decoding from binary code to one-hot code
- Each output is a minterm (product term)
So also a minterm generator
it can be used to implement any 3-input logic function



(1) 3-to-8 line decoder

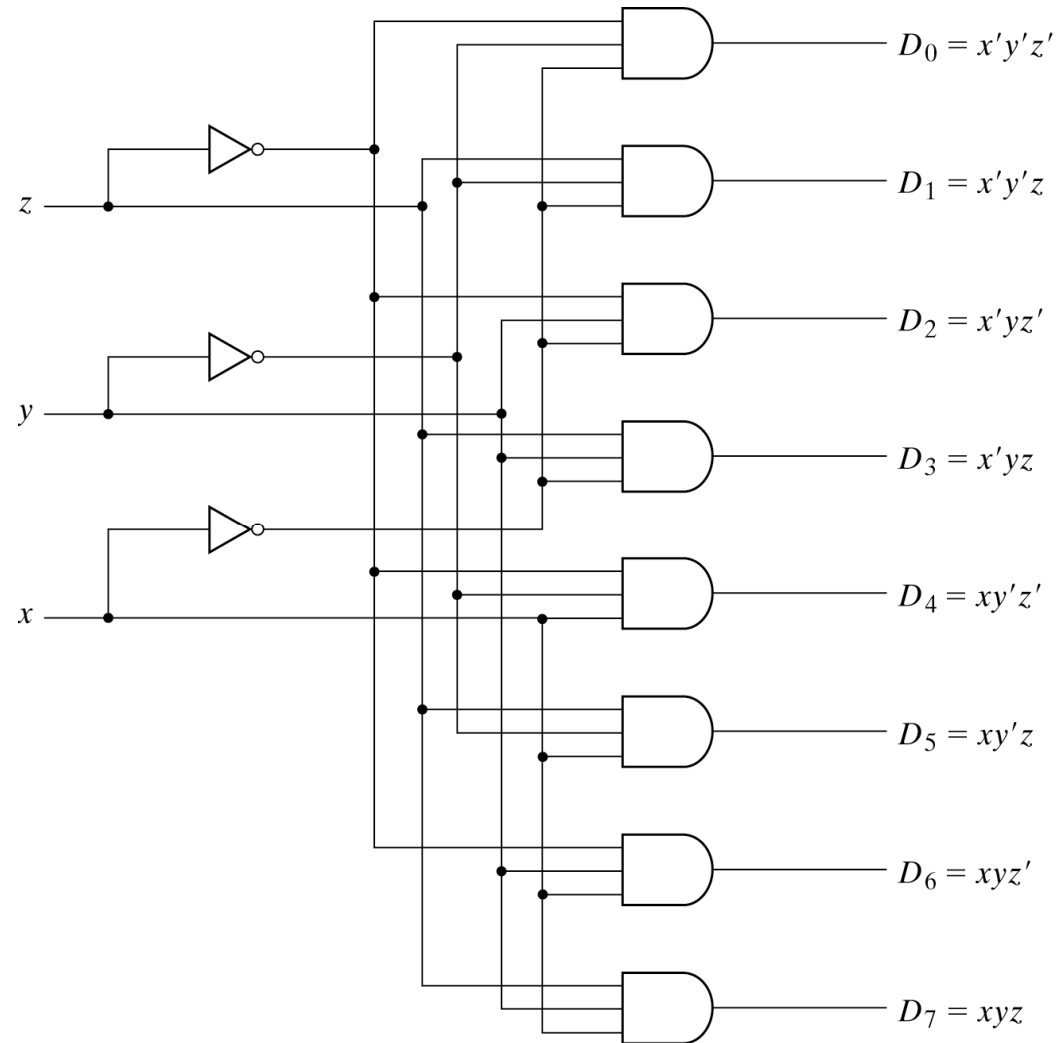


N inputs to 2^N outputs



**All the minterms are available
at outputs
(minterm generator)**

Gate schematic



Minterms

SOP implementation Full Adder

TABLE 6-2

Full-adder truth table.

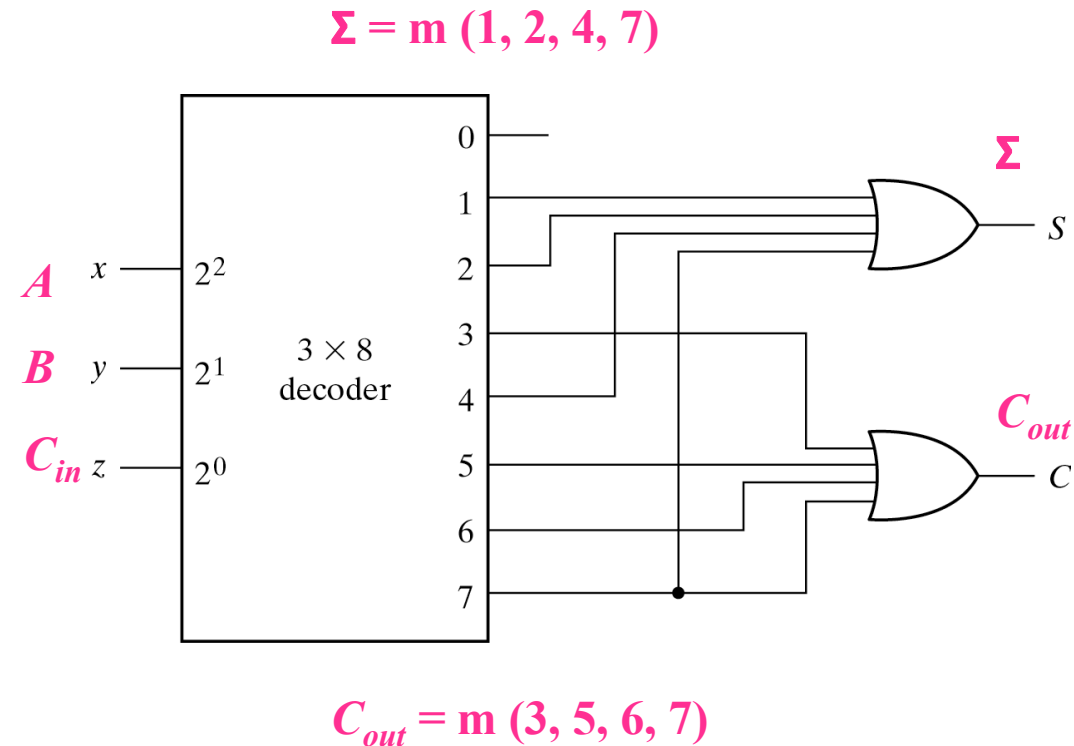
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

Σ = sum

A and B = input variables (operands)



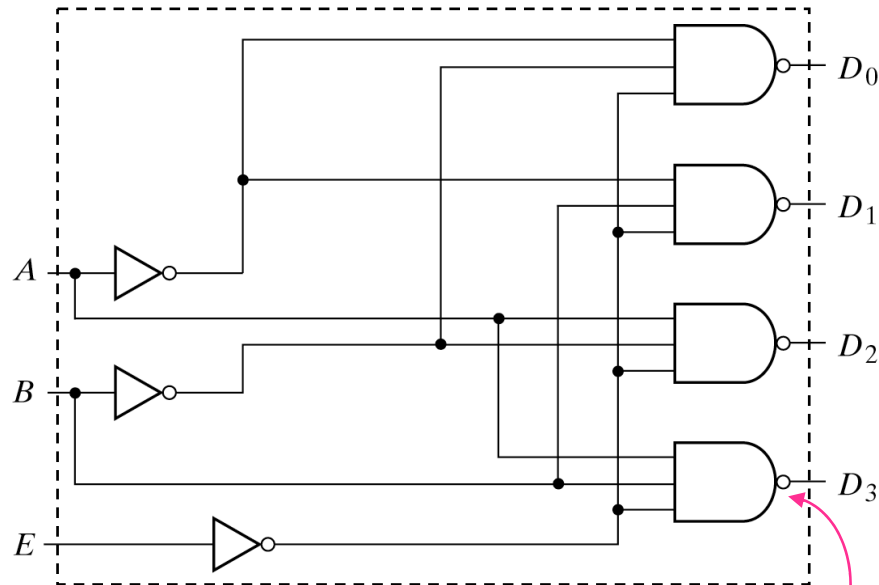
2-to-4 Line Decoder w/ ENABLE



X means it can be either 0 or 1 (don't care).

Outputs are disabled and won't change as long as Enable is 1

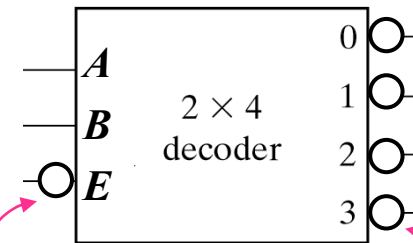
Gate schematic



Bubbles at input or output mean "active LOW"

<i>E</i>	<i>A</i>	<i>B</i>	<i>D</i> ₀	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃
1	<i>X</i>	<i>X</i>	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

Maxterms



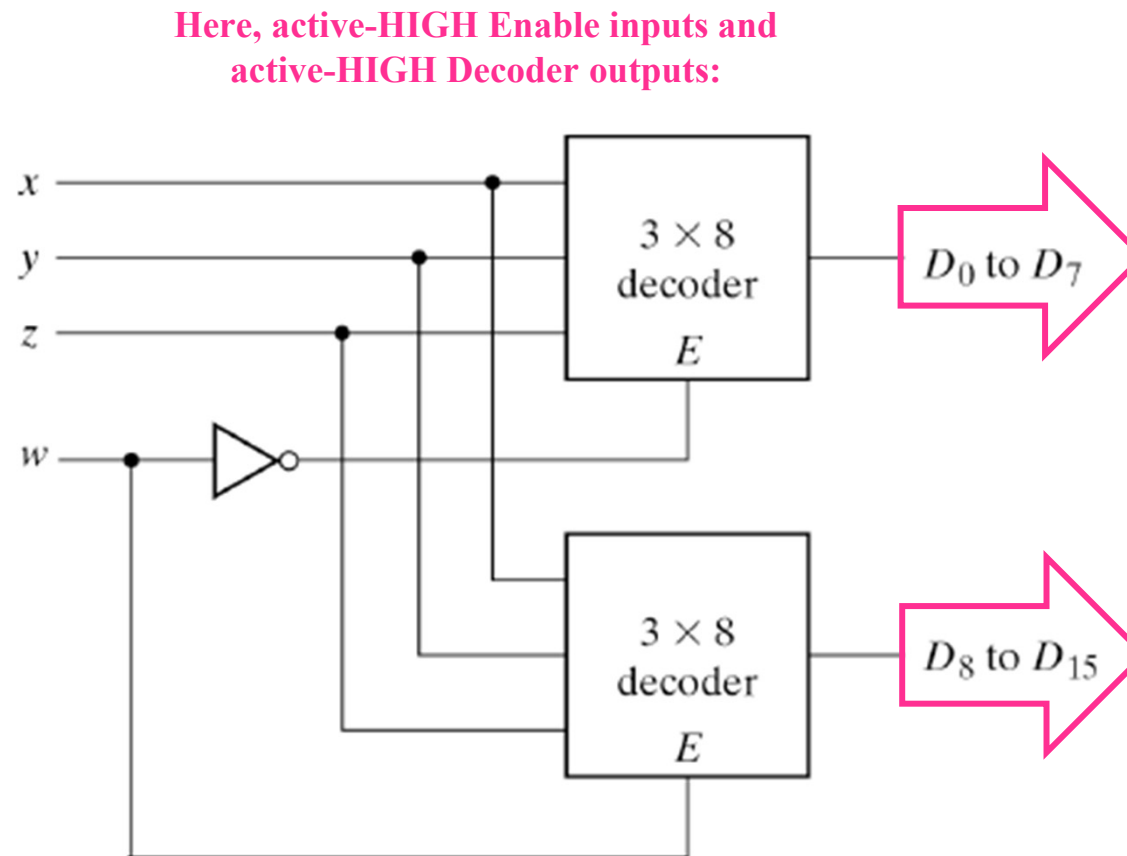
Active-LOW Enable:
Need to assert 0 to
enable the decoder

Active-LOW Outputs:
The outputs that are
asserted 0 are active

(2) Decoder expansion using enable inputs



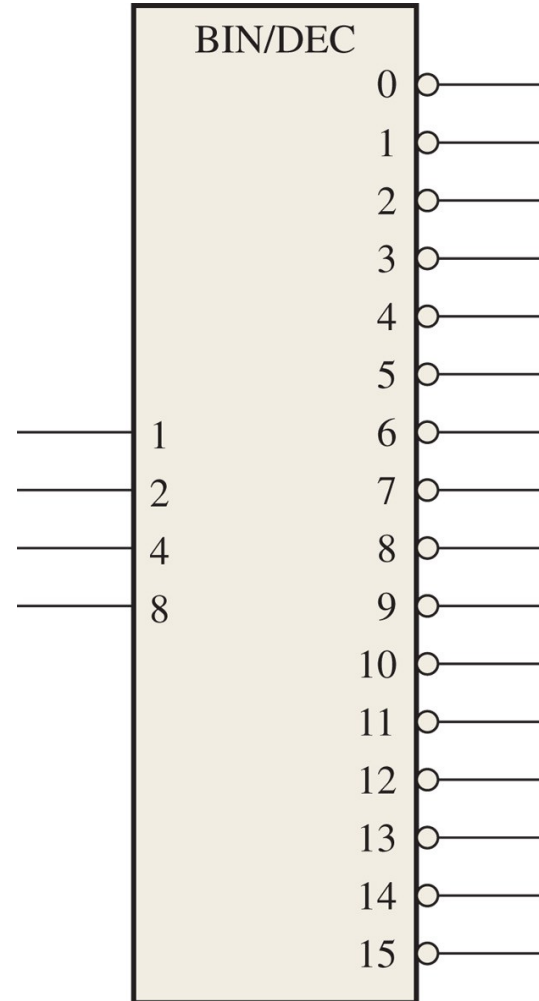
- A 4-to-16 line decoder is built out of two 3-to-8 line decoders featuring Enable inputs
- Enable inputs are dedicated to MSB (w)



wxyz	D(15:8)	D(7:0)
0 000	00000000	00000001
0 001	00000000	00000010
0 010	00000000	00000100
0 011	00000000	00001000
0 100	00000000	00010000
0 101	00000000	00100000
0 110	00000000	01000000
0 111	00000000	10000000
1 000	00000001	00000000
1 001	00000010	00000000
1 010	00000100	00000000
1 011	00001000	00000000
1 100	00010000	00000000
1 101	00100000	00000000
1 110	01000000	00000000
1 111	10000000	00000000

(2) Decoder expansion using enable inputs

4-to-16 (BINARY to DECIMAL) DECODER

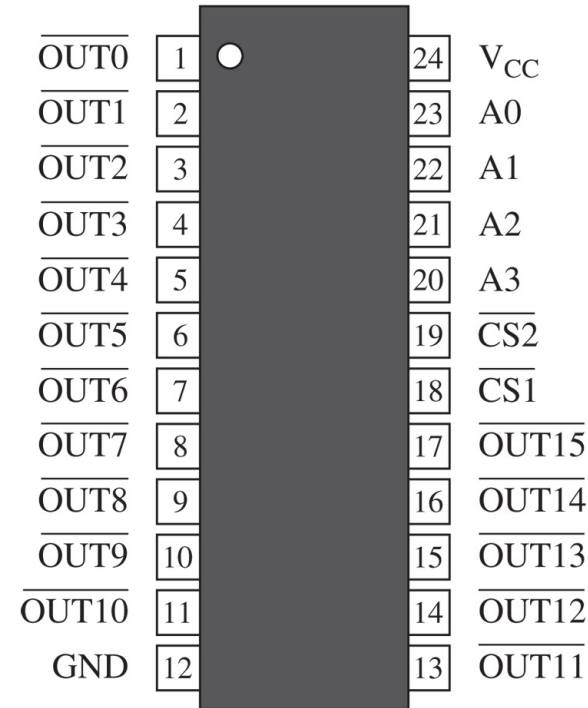


Bubbles at the output terminals indicate that they are “active-LOW” as opposed to “active-HIGH” output

Meaning when an output is asserted 0 “LOW” that output is active

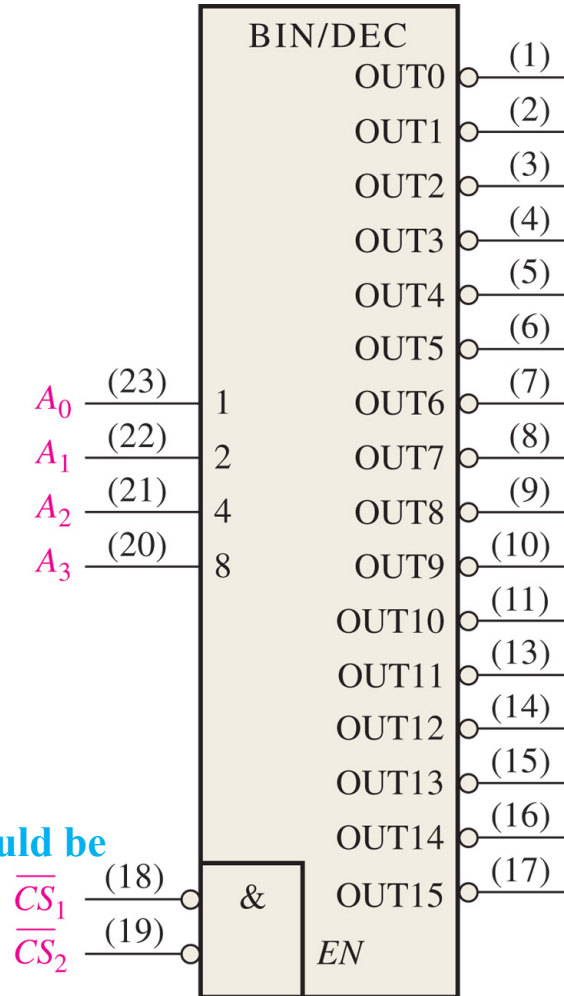
For this decoder, all the outputs will be de-asserted 1 “HIGH” except the output which is the equivalent of binary input supplied

(3) 74HC154 4-to-16 decoder implementation



EN input depends on chip select inputs $\overline{CS}_1$ and $\overline{CS}_2$
To enable the chip ($EN = 1$) both chip select inputs should be grounded (Active “LOW”)

(a) Pin diagram

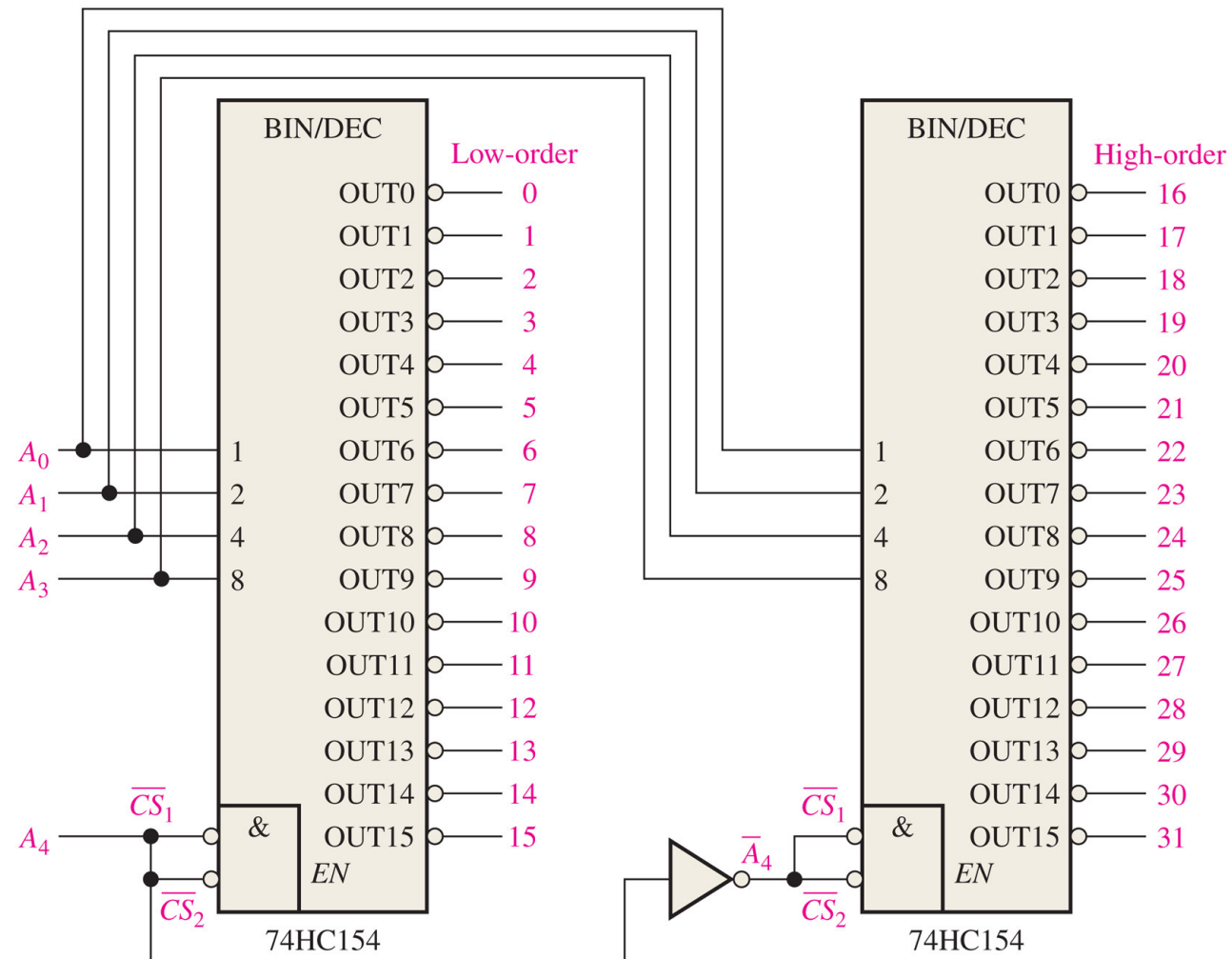


(b) Logic symbol

(3) 74HC154 4-to-16 decoder implementation



A 5-bit Decoder using 74HC154s



(1) BCD-to-decimal decoder (74HC42)

- BCD code refers to “Binary Coded Decimal”
- Ten decimal digits expressed in binary using 4 bits

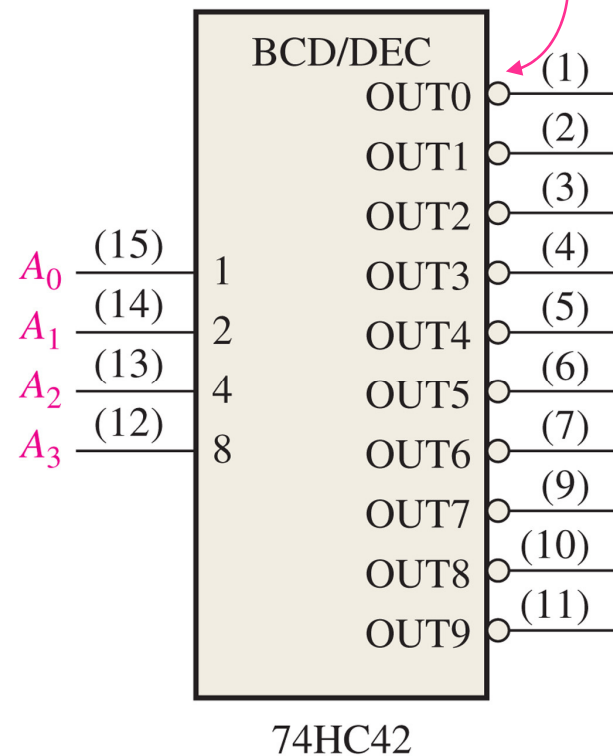
TABLE 6-6

Decimal Digit	BCD Code			
	A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

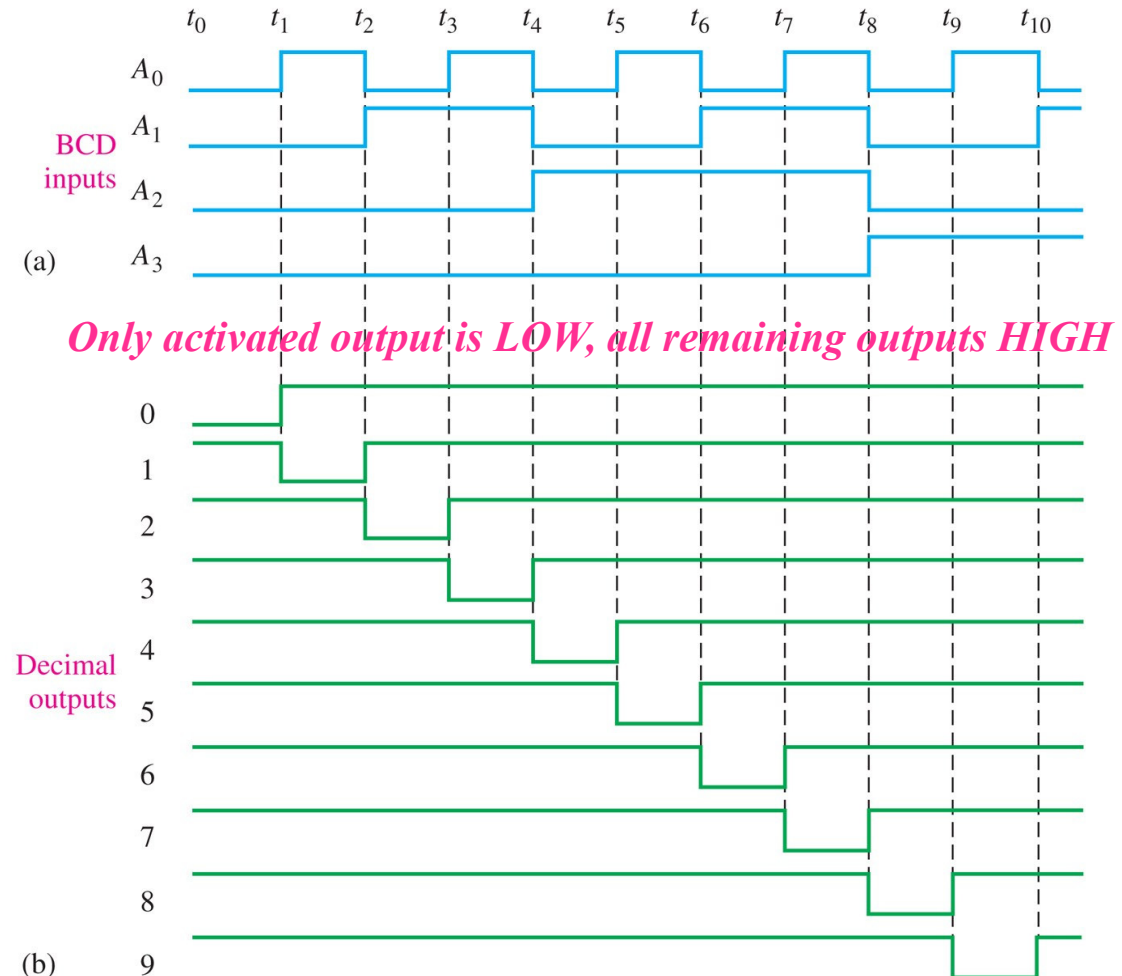
(1) BCD-to-decimal decoder (74HC42)

The 74HC42 BCD-to-decimal decoder

*Bubbles indicate Active “LOW” outputs:
Output binary 0 means the output is active*



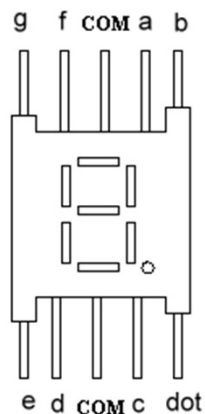
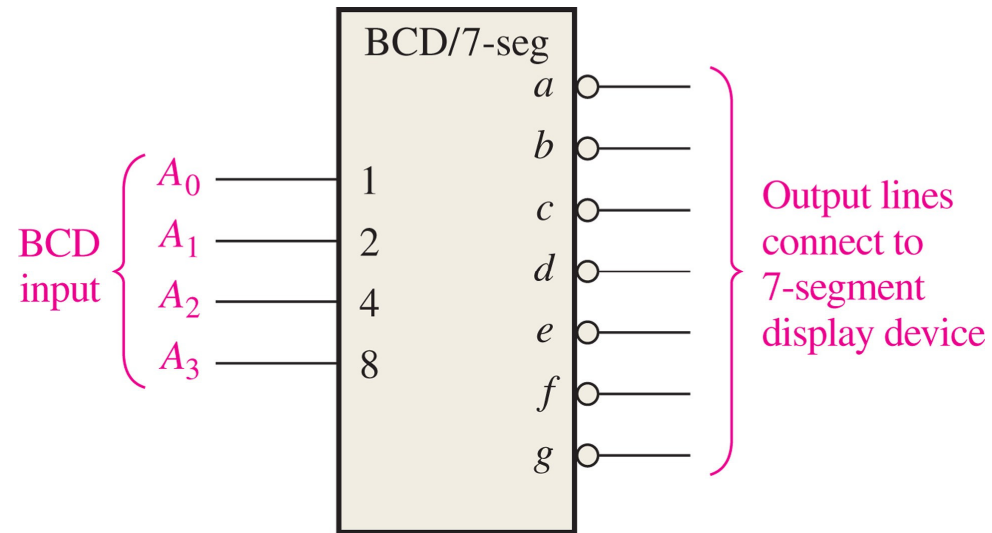
accepts the BCD code on its inputs and produce a decimal output



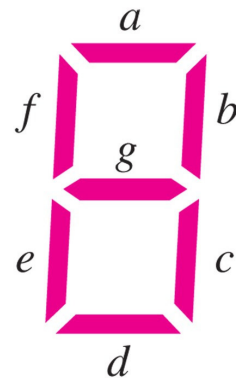
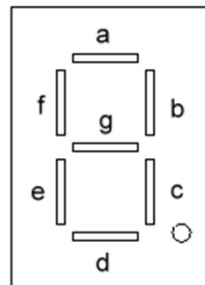
(2) BCD-to-7 segment decoder



It is used to drive 7-segment displays



Seven-Segment Display



(a) Segment arrangement

TABLE 4-14

Active segments for each of the five letters used in the system display.

Letter	Segments Activated
A	<i>a, b, c, e, f, g</i>
b	<i>c, d, e, f, g</i>
C	<i>a, d, e, f</i>
d	<i>b, c, d, e, g</i>
E	<i>a, d, e, f, g</i>

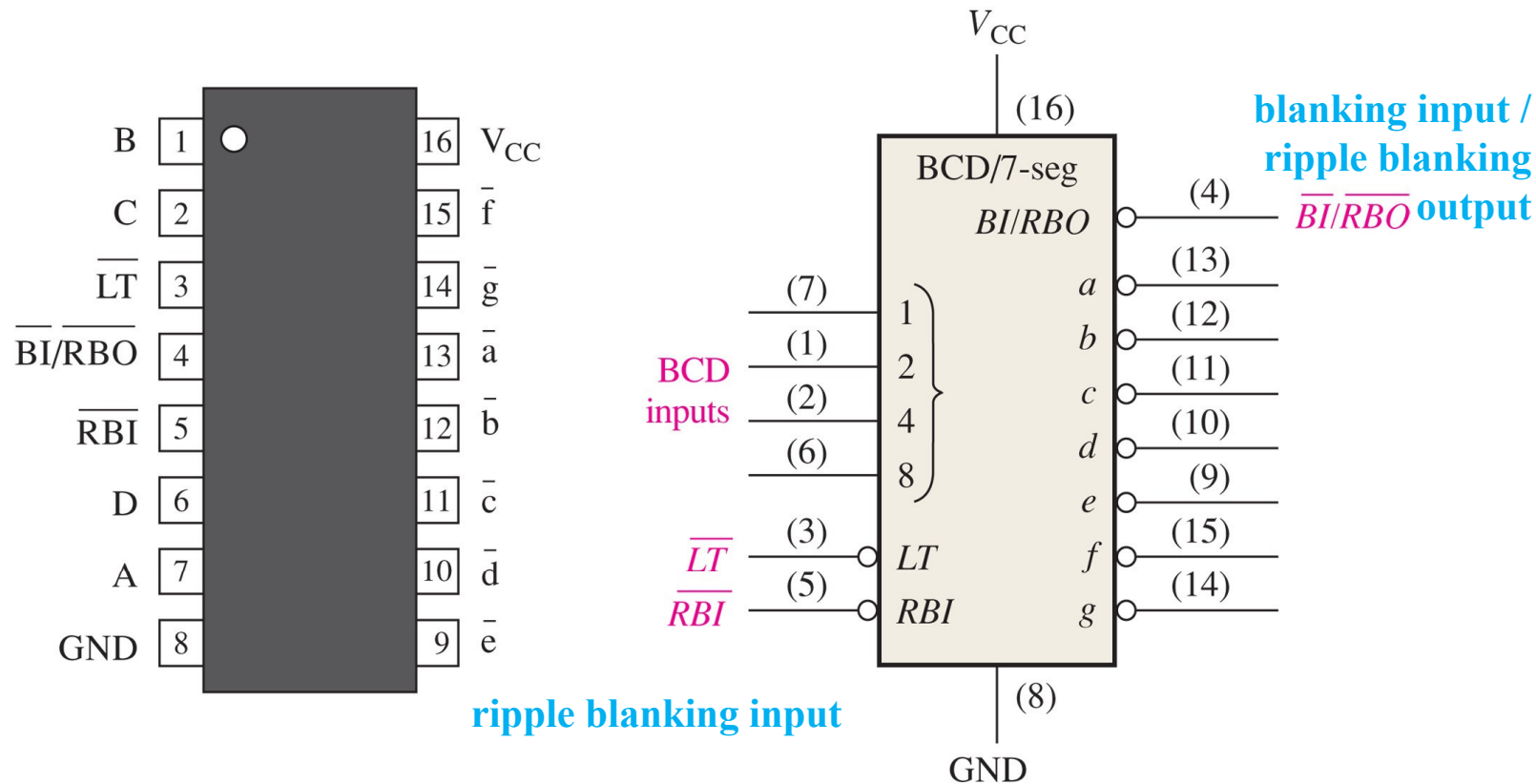


(b) Formation of the ten digits and certain letters

(2) BCD-to-7 segment decoder



The 74HC47 BCD-to-7-segment decoder/driver



(a) Pin diagram

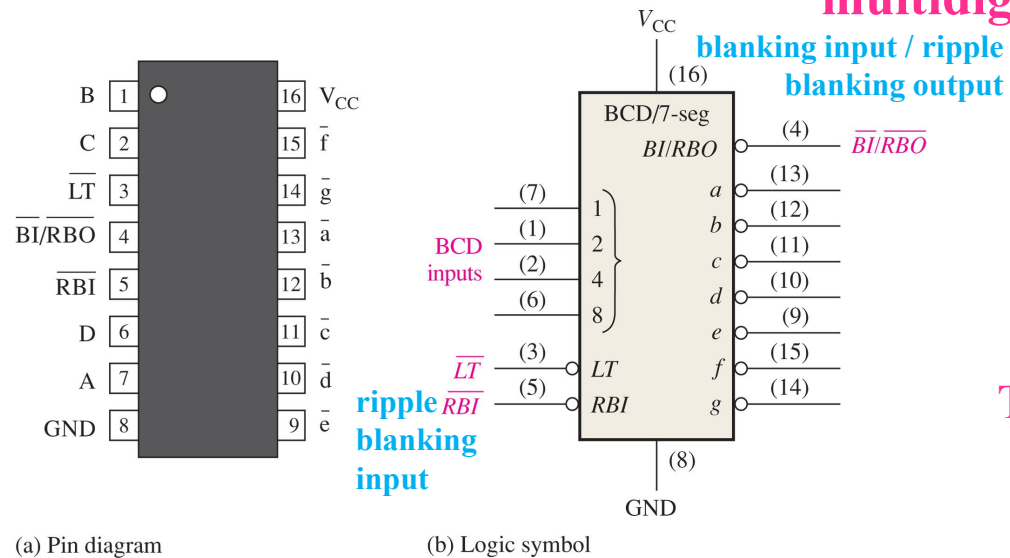
(b) Logic symbol

$\overline{LT}$ input is for Lamp Test to check for any burned out segment.

All the seven segments are turned when $\overline{LT}$ input is LOW and the $\overline{BI/RBO}$ is HIGH

(2) BCD-to-7 segment decoder

The 74HC47 BCD-to-7-segment decoder



$\overline{RBI}$ and $\overline{RBO}$ are used for zero suppression in multidigit displays to blank out unnecessary zeros.

Example:

A 6-digit display will show the number as 006.400 instead of 6.4 if the zeros are not blanked out (suppressed)

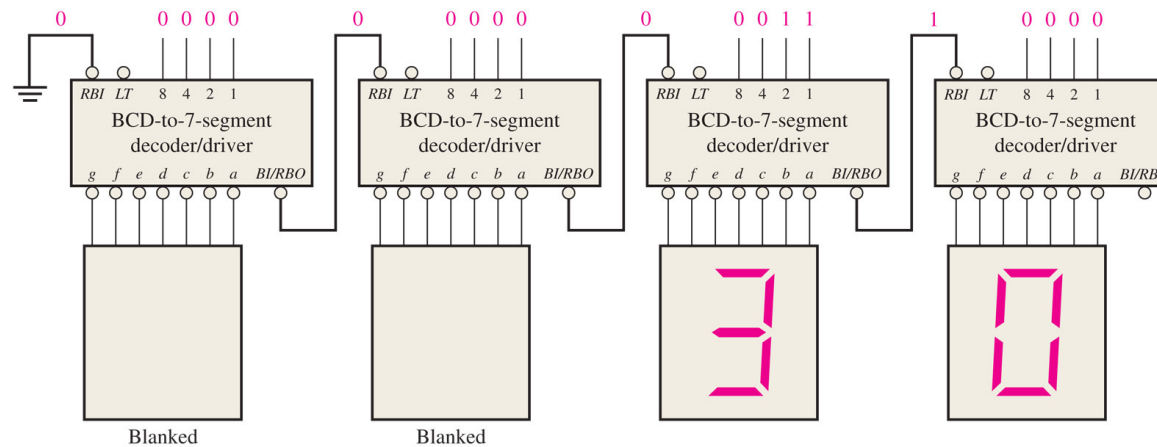
The $\overline{BI}$ is not involved in zero suppression but, when LOW, overrides all other inputs and all segment outputs go HIGH (nonactive).

A LOW $\overline{RBI}$ and LOW BCD inputs (code 0000) produce a LOW $\overline{RBO}$ and render all segment outputs of the decoder nonactive (HIGH) causing the display to go blank.

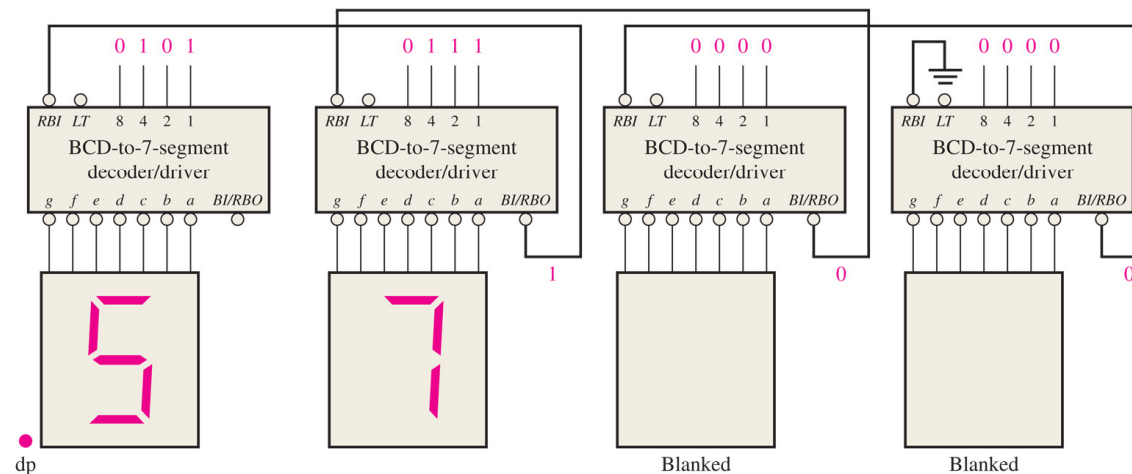
(2) BCD-to-7 segment decoder

Zero Suppression

A LOW $\overline{RBI}$ and LOW BCD inputs (code 0000) produce a LOW $\overline{RBO}$ and render all segment outputs of the decoder nonactive (HIGH) causing the display to go blank



(a) Illustration of leading zero suppression



(b) Illustration of trailing zero suppression

Main Content

1. BCD-to-Binary Conversion
2. Gray Code

1. BCD-to-Binary Conversion

(1) Weight-based conversion method

	Tens Digit				Units Digit			
Weight:	80	40	20	10	8	4	2	1
Bit designation:	B_3	B_2	B_1	B_0	A_3	A_2	A_1	A_0

The binary equivalent of each BCD bit is a binary number representing the weight of that bit within the total BCD number. This representation is given in Table 6–7.

Binary representations of BCD bit weights.

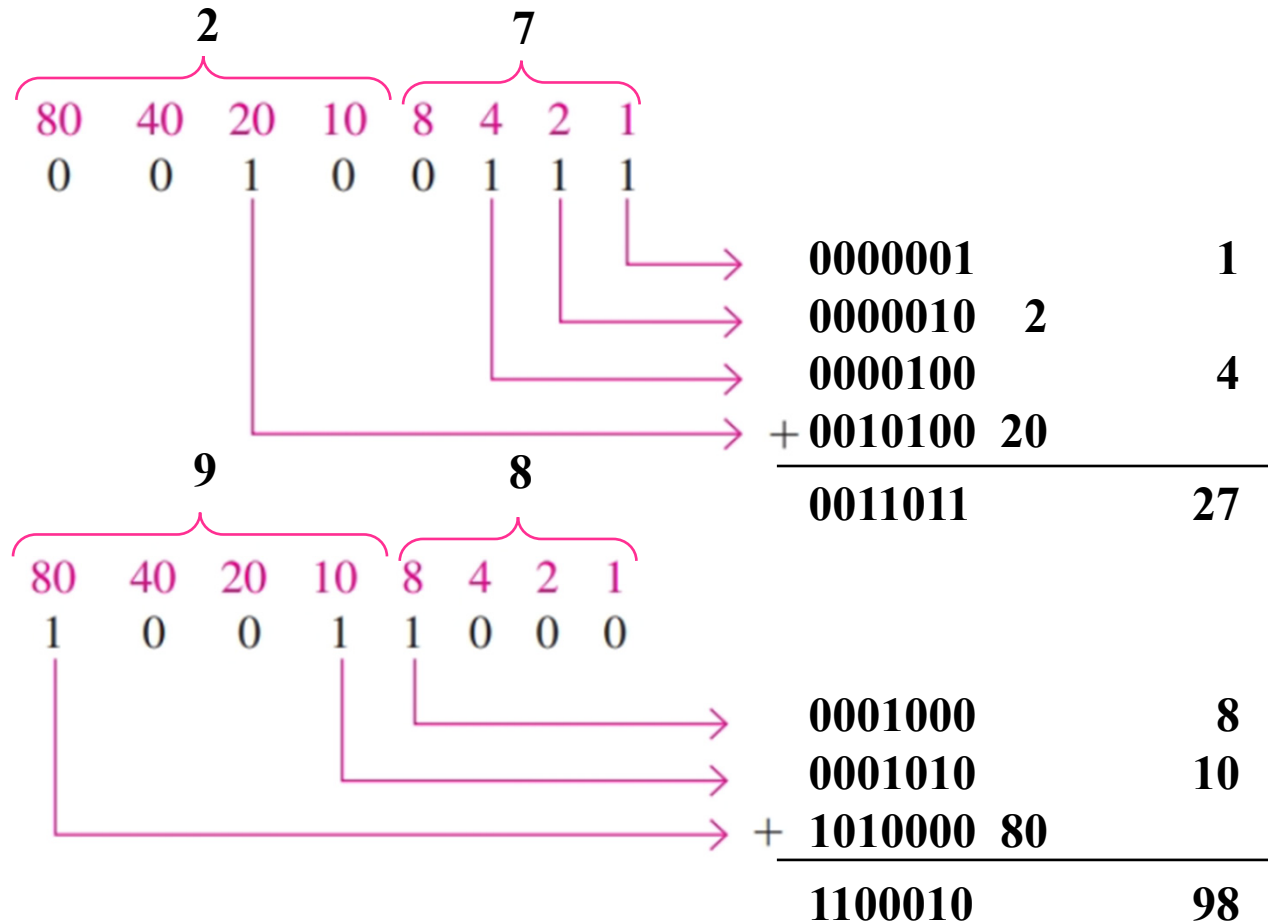
BCD Bit	BCD Weight	Binary Representation						
		(MSB) 64	32	16	8	4	2	(LSB) 1
A_0	1	0	0	0	0	0	0	1
A_1	2	0	0	0	0	0	1	0
A_2	4	0	0	0	0	1	0	0
A_3	8	0	0	0	1	0	0	0
B_0	10	0	0	0	1	0	1	0
B_1	20	0	0	1	0	1	0	0
B_2	40	0	1	0	1	0	0	0
B_3	80	1	0	1	0	0	0	0

Adding the binary representations for the weights of all the 1s in the BCD number gives the binary number that corresponds to the BCD number.

1. BCD-to-Binary Conversion

Example (BCD $27_{10} \rightarrow 00011011_2$)

Convert the BCD numbers 00100111 (decimal 27) and 10011000 (decimal 98) to binary



(1) Weight-based conversion method

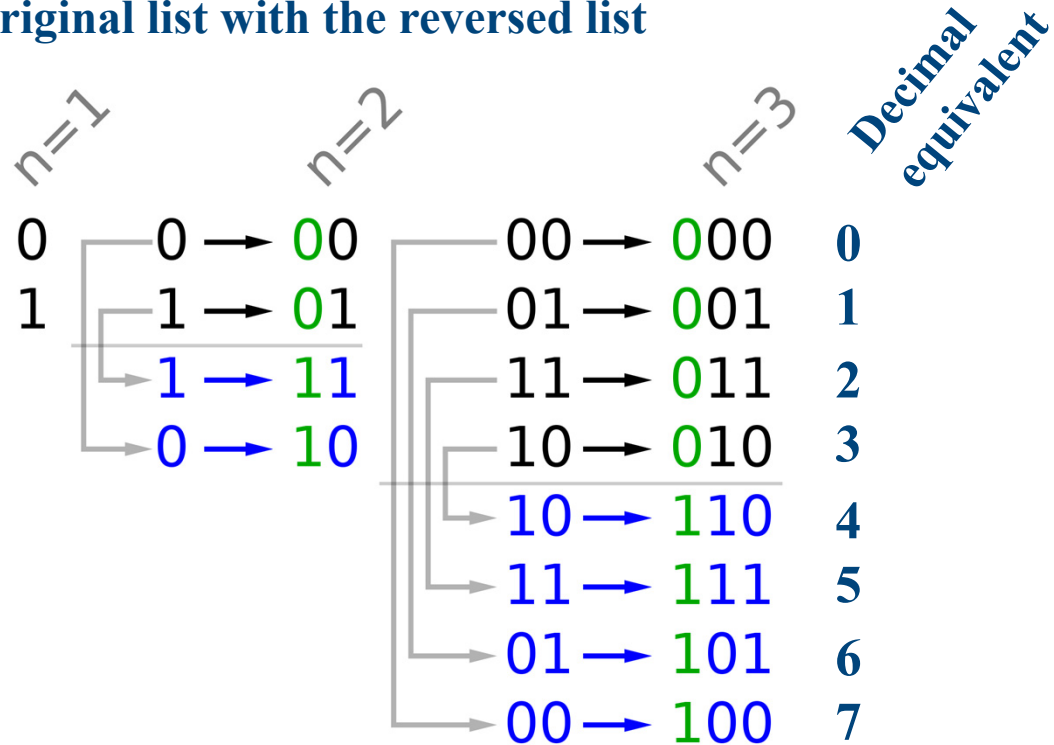
- Adding the binary representations for the weights of all the 1s in the BCD number gives the binary number that corresponds to the BCD number
- The binary numbers representing the weights of the BCD bits are summed to produce the total binary number
- Thus, BCD-to-binary code conversion can be done using adder circuits. Adder circuits become BCD-to-Binary decoder

(1) Gray code properties (single-bit transitions)

- **Unweighted! Not an arithmetic code**
 - no specific weights assigned to the bit positions.
- **Exhibits only a single bit change from one code word to the next in sequence**
 - Important property in many applications to avoid error risk which increases with the number of bit changes between adjacent numbers in a sequence

(2) Reflective construction method for n -bit codes

- Also known as reflected binary code (RBC) or reflected binary (RB) because of the way it's constructed
- Recursively generated from the list for $n - 1$ bits by
 - reflecting the list (i.e. listing the entries in reverse order),
 - prefixing the entries in the original list with a binary 0,
 - prefixing the entries in the reflected list with a binary 1, and then
 - combining the original list with the reversed list



(2) Reflective construction method for n -bit codes

- Notice the single-bit change between successive code words
- For instance, in going from decimal 3 to decimal 4,
 - the Gray code changes from 0010 to 0110, while the binary code changes from 0011 to 0100, a change of three bits
 - the only bit change in the Gray code is in the third bit from the right: the other bits remain the same

Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

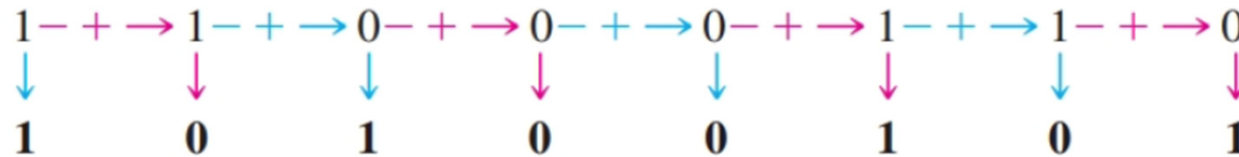
(3) Binary $\leftrightarrow$ Gray conversion logic

EXAMPLE 2-37

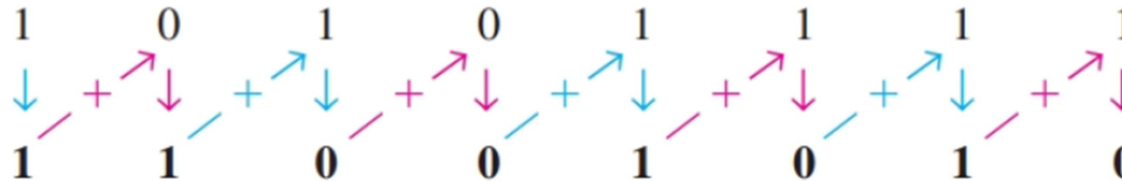
- (a) Convert the binary number 11000110 to Gray code.
- (b) Convert the Gray code 10101111 to binary.

Solution

- (a) Binary to Gray code:



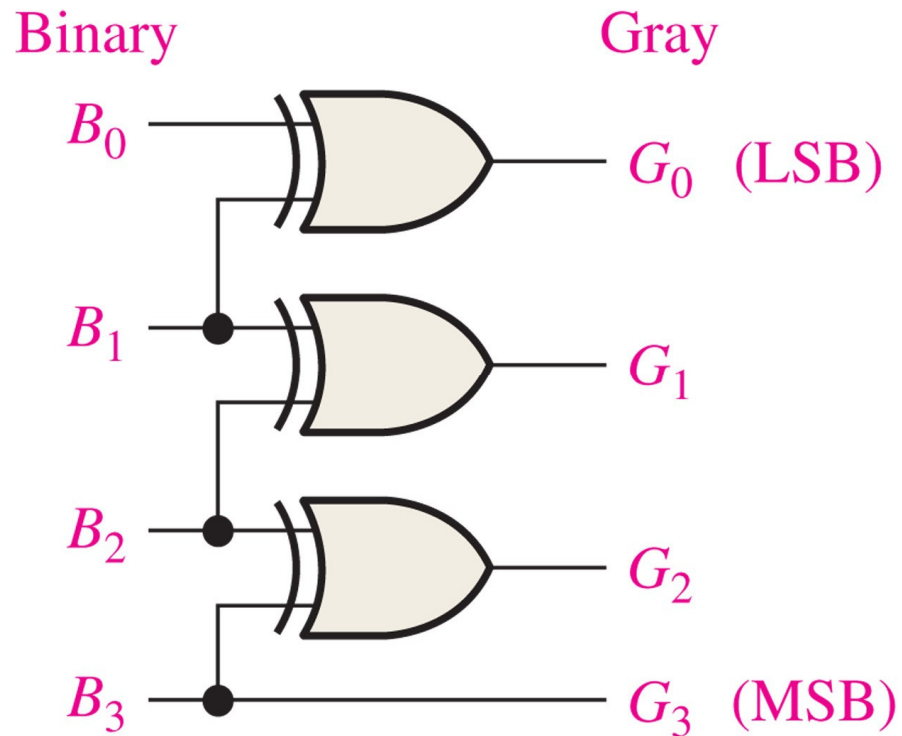
- (b) Gray code to binary:



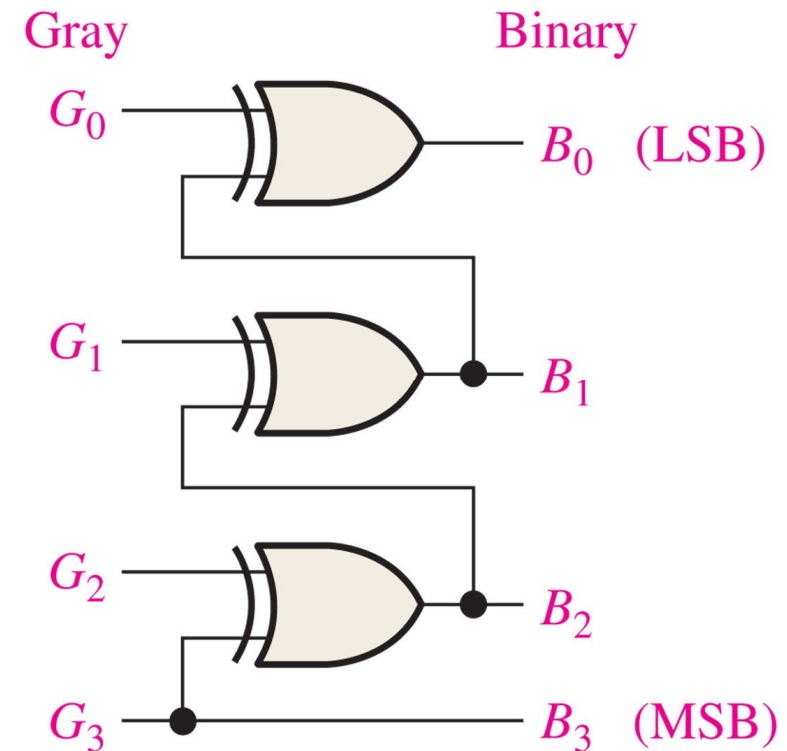
2. Gray Code

(3) Binary $\leftrightarrow$ Gray conversion logic

Four-bit binary-to-Gray conversion logic



Four-bit Gray-to-binary conversion logic



2. Gray Code

(3) Binary $\leftrightarrow$ Gray conversion logic

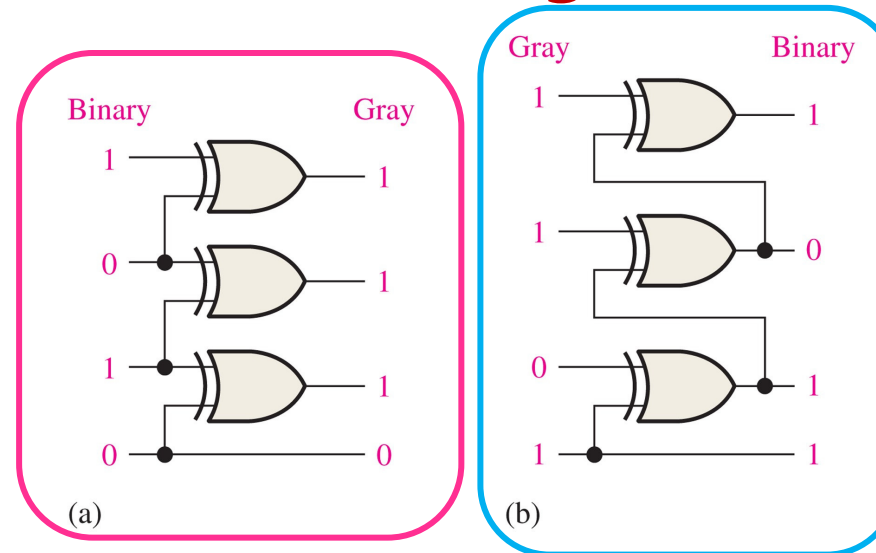


TABLE 2-6

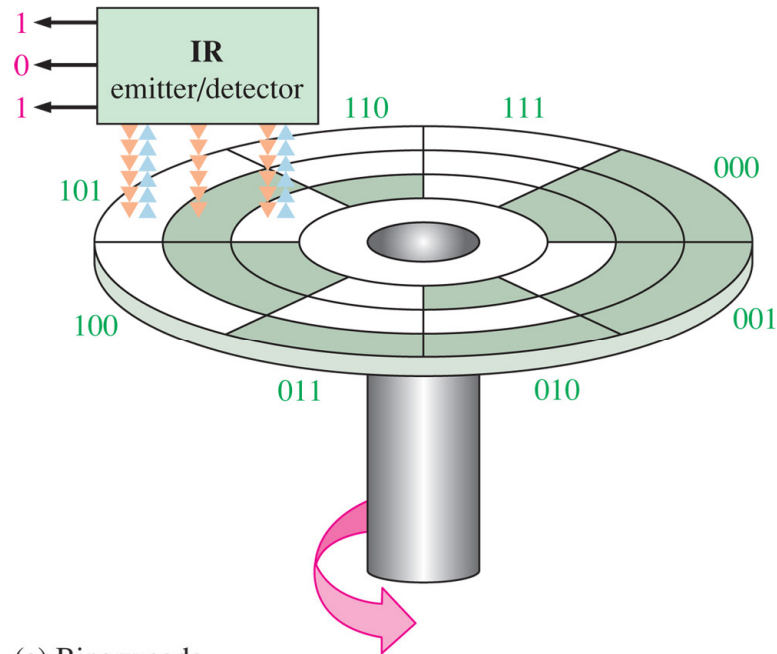
Four-bit Gray code.

Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

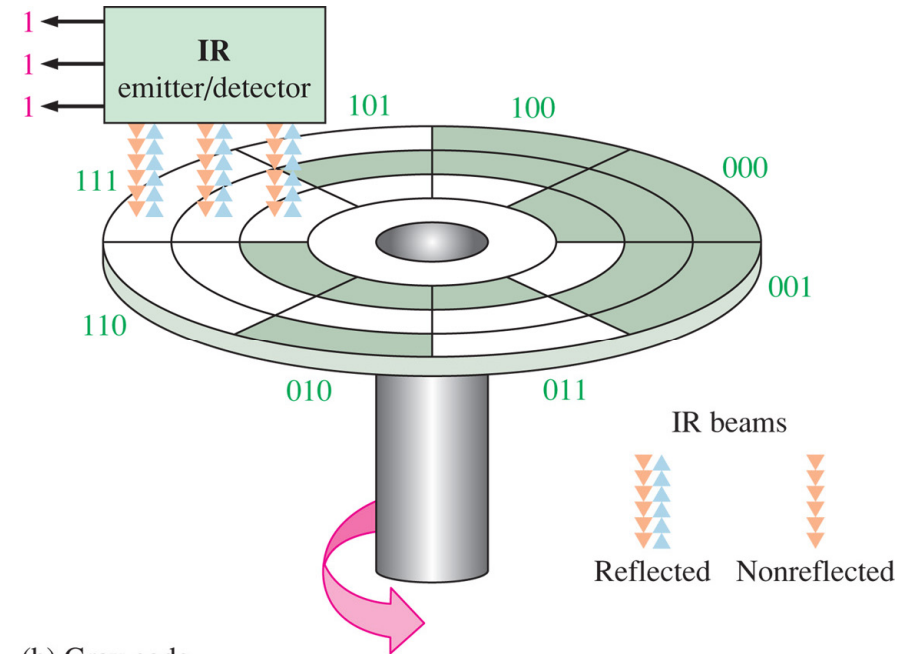
(4) Application in shaft encoders (error-resistant transitions)

A simplified illustration of how the Gray code solves the error problem in shaft position encoders.

Three bits are shown to illustrate the concept, although most shaft encoders use more than 10 bits to achieve a higher resolution.



(a) Binary code



(b) Gray code

(4) Application in shaft encoders (error-resistant transitions)

- The IR emitter/detector sends infrared beams from a fixed position to each track on the rotating shaft featuring reflective segments
- It detects the presence of reflected beams and produces a 3-bit code.
 - 1 means reflected beam
 - 0 means no reflected beam
- Note the shaft surface has eight sectors (like a pizza slice) and each sector represent 3-bit Gray Code based on reflective and non-reflective segments



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

